
Grounding Driving VLA via Inverse Kinematics

Junsung Park, Hyunjung Shim
Korea Advanced Institute of Science and Technology
KimJaeChul AI Graduate School
18, Taebong-ro, Seocho-gu, Seoul, South Korea
{jshackist, kateshim}@kaist.ac.kr

Abstract

Counterfactual obstacle insertion exposes blind planning in representative Driving VLAs: their trajectories remain largely unchanged despite a vehicle appearing directly ahead. An empirically calibrated bias–variance analysis identifies how ego-predictable training examples admit a shortcut through ego status and commands. We address this failure with an inverse-kinematics-inspired architecture that anchors trajectory decoding on current and predicted future visual states. First, a next visual state prediction objective that requires the LLM to predict the future visual scene provides dense visual supervision and suppresses shortcut paths. Second, a separate Inverse Kinematics Network (a cross-attention-based conditional diffusion model) that takes only the current and future visual states as input is designed to suppress reliance on ego status and textual shortcuts during trajectory decoding. Our 0.5B model achieves state-of-the-art trajectory planning performance on NAVSIM-v1, NAVSIM-v2, and nuScenes using cameras alone, without LiDAR or a BEV encoder. Against the matched OpenDriveVLA* baseline, it improves NAVSIM PDMS and EPDMS by 19.0 and 20.4 points, respectively. Controlled ablations and intervention analyses support improved use of visual features, with additional benefits from future-state prediction in dynamic situations such as turning.

1 Introduction

In autonomous driving, Vision-Language-Action models (Driving VLA) are rapidly emerging as a unified policy that takes camera images and textual commands as input and directly outputs future driving trajectories [17, 52]. Unlike conventional autonomous driving pipelines, which optimize perception, prediction, and planning as separate modules, this approach unifies the three stages into a single end-to-end policy within a VLM backbone, offering benefits in simplicity and scalability.

Camera conditioning alone does not ensure visually grounded planning. Our counterfactual experiments expose *blind planning* in representative Driving VLAs: inserting a vehicle directly ahead produces little change in their predicted trajectories. As shown in Fig. 1, representative existing Driving VLAs [52, 44, 2] barely change their trajectory even when a vehicle is synthesized in front of the ego vehicle, maintaining the same straight path as before the synthesis, which exemplifies blind planning by definition. Building on prior observations of ego-status shortcuts [27], we quantify this failure through controlled visual interventions that hold non-visual inputs fixed.

We identify a structural opportunity for blind planning in the coupling between trajectory supervision and decoder inputs. Drawing an analogy to inverse kinematics – where a trajectory is anchored by both a start and a target configuration – we observe that supplying a future visual state $V_{t+\Delta}$ alongside the current visual state V_t provides the model an explicit reason to ground its trajectory in visual features. In the direct-prediction formulation, the decoder can fit ego-predictable trajectories through ego status and commands without reconstructing the future visual scene. Our bias–variance analysis characterizes this shortcut under an empirically calibrated majority-regime assumption. This is a

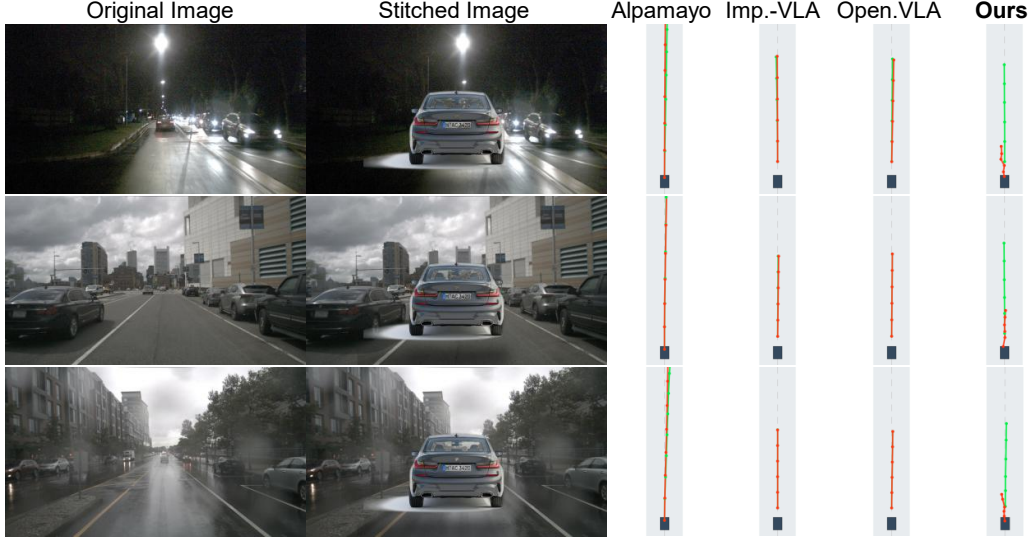


Figure 1: Counterfactual obstacle stitching on nuScenes val. Left columns: original front-camera image and the same image with a near-range vehicle sprite stitched in front of the ego. Right columns: BEV trajectories before (green) and after (red) stitching for existing VLAs and our model. While existing VLAs’ trajectory remains nearly invariant to the visual input, whereas our model produces a distinct slowdown/avoidance reaction in response to the stitched obstacle, demonstrating that visual features causally influence its trajectory output.

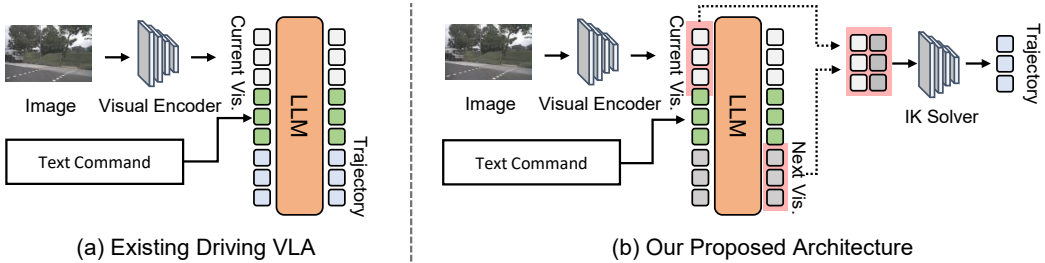


Figure 2: Existing Driving VLA (a) vs. our model (b). Existing models decode the trajectory directly from the current visual tokens V_t , ego status E , and text command C . In contrast, ours requires the LLM to predict a future visual state $\hat{V}_{t+\Delta}$ and decodes the trajectory through an IK Network conditioned only on the pair $(V_t, \hat{V}_{t+\Delta})$, so that ego status and text commands do not directly enter the trajectory decoder.

typical shortcut-learning failure in end-to-end training [27]: when the visual scene is absent from the output space, the model is rarely rewarded for exploiting it.

This motivates two complementary interventions: on the supervision side, the model is never asked to reason about future visual scenes, and on the architecture side, the trajectory decoder is free to bypass visual tokens entirely and rely on ego status and text commands. We address both at once, as illustrated in Fig. 2(b). First, we separate the trajectory decoder into a dedicated Inverse Kinematics Network (IK Network), a bottleneck conditioned only on the pair of the current visual token V_t and the future visual token $\hat{V}_{t+\Delta}$ predicted by the backbone. This removes the direct ego-status and command pathway to trajectory decoding and makes the current/predicted visual-state pair the action expert’s conditioning interface. We use the term “inverse kinematics” in a stylistic sense, following its spirit rather than the strict vehicle-dynamics formulation: a trajectory is regressed from the observed change between two visual states. Second, for this bottleneck to be meaningful, $\hat{V}_{t+\Delta}$ must be explicitly produced by the backbone, so we additionally introduce a next-state prediction objective that places the visual scene back into the output space, addressing the supervision-side gap.

Through these architectural and training modifications, the primary target of this paper is the recovery of visual grounding in Driving VLA.

To this end, we conduct a series of validations including GradCAM, counterfactual obstacle stitching, and per-sample statistical analyses, demonstrating both quantitatively and qualitatively that the model indeed comes to rely on visual features when determining trajectories. Our 0.5B model achieves state-of-the-art trajectory planning performance on NAVSIM-v1, NAVSIM-v2, and nuScenes. It improves over the matched OpenDriveVLA* baseline by 19.0 PDMS and 20.4 EPDMS on NAVSIM while using cameras alone, without LiDAR or a BEV encoder. More broadly, these results hint at a remedy for a common pitfall in end-to-end learning: models tend to ignore any input modality that is not part of the supervision target.

Our contributions are summarized as follows.

- **Theoretical Analysis:** We demonstrate via bias–variance decomposition that *blind planning* is a theoretically derived loss-optimal shortcut rather than a training failure, caused by visual-feature penalization in standard trajectory losses.
- **Architectural Remedies:** To restore visual grounding, we introduce a next-state prediction objective and an IK-Network bottleneck that forces the model to process ego-status through visual features.
- **Superior Efficiency:** Through counterfactual obstacle stitching, and per-sample statistical analyses, the proposed design is shown to actually recover visual grounding, and as a result, achieves driving performance on NAVSIM v1, NAVSIM v2 and nuScenes comparable to VLAs more than an order of magnitude larger (7B–8B) using only a 0.5B model.

2 Related Works

2.1 End-to-end Planning

End-to-end planning models take vehicle information such as the input image and ego status, and predict the trajectory along which the vehicle will proceed. ST-P3 [13] jointly optimizes perception/prediction/planning in vision-based end-to-end autonomous driving via BEV transformation and spatial-temporal feature learning. VAD [19] represents the driving scene with a vectorized representation of agents and maps, simultaneously enhancing the efficiency and safety of planning. UniAD [15] integrates perception–prediction–planning into a single network and strengthens inter-module coordination through a planning-oriented design and unified queries. InsightDrive [40] organizes the attention required for planning through a language-guided scene representation that highlights “regions important for driving” in language. Ego-MLP [51] analyzes how ego status can be a strong shortcut in open-loop planning and shows the possibility of high performance based solely on ego. Building on this, Li *et al.* [27] show that a simple MLP using only physical state, without vision/point cloud, can be competitive on existing open-loop metrics, pointing out the loopholes of the evaluation metrics. GenAD [21] presents a generative E2E paradigm that strengthens planning by predicting future scene/interaction changes from a “generative modeling” perspective. DiffusionDrive [28, 54] significantly reduces denoising steps with a truncated diffusion policy using multi-intent anchors and a cascade decoder, improving real-time multimodal trajectory planning performance on NAVSIM. GoalFlow [46] constrains the generative process with a goal point and combines scene-aware goal scoring with flow matching to efficiently produce high-quality multimodal trajectories with very few generation steps.

2.2 Driving VLA

Driving VLA models, like end-to-end planning models, take vehicle information such as input images and ego status, predict the trajectory along which the vehicle will proceed, and additionally perform scene description. GPT-Driver [31] reformulates motion planning as language modeling and produces trajectories by generating coordinate tokens with a GPT-based model. DriveGPT4 [47] proposes an interpretable end-to-end driving MLLM that jointly generates control signals and explanations through vision-language instruction tuning. DriveLM [38] presents data and benchmarks that elicit multi-step reasoning via graph-structured VQA, enhancing the generalization and interactivity of driving decision making. DriveVLM [41] combines scene description, analysis, and hierarchical

planning to strengthen reasoning-based planning in long-tail situations. RDA-Driver [16] introduces a reasoning-decision alignment constraint to reduce the discrepancy between the chain-of-thought reasoning and the final decision, improving planning reliability. DME-Driver [9] separates a VLM-based “Decision Maker” and a 3D-perception-based “Executor” to combine explainable decision making with stable control generation. EMMA [17] processes camera input in language space with a multimodal LLM, directly mapping it to various driving outputs such as trajectories, objects, and road graphs. OpenEMMA [45] is a lightweight open-source MLLM that takes ego-history and the front camera as input and predicts speed/curvature via chain-of-thought, integrating them into trajectory planning. OpenDriveVLA [52] presents a VLA that combines 2D/3D instance-aware tokens, ego state, and language commands to produce spatially grounded driving actions/trajectories. Impromptu VLA [2] provides large-scale VLA data and open weights focused on corner cases, improving the robustness and open/closed-loop performance of VLAs. GPVL [21] reduces the gap between perception and language through vision-language pretraining on BEV, and proposes a generative planning approach that autoregressively generates decisions and trajectories. OmniDrive [42] builds a vision-language dataset that covers 3D driving understanding/planning based on counterfactual reasoning, strengthening VLM alignment and evaluation. All such existing driving VLAs take an image as the initial condition and immediately perform trajectory prediction. We point out that this approach of existing driving VLAs, which lacks a terminal condition, is problematic from an inverse kinematics perspective, and we present a new architecture that addresses this problem.

2.3 Driving World Models

Driving World Models enable next state prediction for driving scenes, supporting tasks such as the simulation of driving scenarios. LAW [22] predicts future latent features in a self-supervised manner conditioned on ego actions/trajectories, strengthening end-to-end driving without label dependence. Drive-WM [43] is a controllable multiview world model that generates future images for various maneuvers and supports safe planning with image-based rewards. GAIA-1 [11] presents a token-based generative world model that takes video, text, and action as input and generates controllable driving scenarios. GAIA-2 [34] proposes a controllable multi-view diffusion world model that simultaneously addresses multi-agent interactions and multi-camera consistency through structured conditioning. Most recently, DriveVLA-W0 [23] integrates future world prediction as an auxiliary objective for the VLA to enhance data scaling.

Distinction from DriveVLA-W0. Although both methods predict future states, our contribution changes the structure and causal role of future prediction beyond an input-restriction ablation. In DriveVLA-W0 [23], the predicted future serves as a reasoning token within an autoregressive decoder, while trajectory generation remains autoregressive with direct access to ego status and text commands. Our next visual state prediction (NSP), by contrast, supplies the boundary conditions of a separate action expert: the IK Network, a conditional diffusion decoder conditioned exclusively on the current and predicted next visual-state pair $(V_t, \hat{V}_{t+\Delta t})$. To our knowledge, this inverse-kinematics formulation, which anchors action decoding on a current/next visual-state pair, has not previously been explored in driving VLAs. Mechanistically, future prediction becomes a required conditioning input at inference, whereas an auxiliary prediction objective alone leaves the trajectory decoder free to exploit the direct ego-status and command pathway. The trajectory-loss analysis in Eqs. (26) to (28) illustrates this distinction under its idealized assumptions: with direct access to (E, C) , the decoder can fit the bias term through these inputs alone, leaving visual collapse as a valid optimum; a decoder that ignores all variation in both visual-state inputs cannot fit a varying target mean. The representation probes and controlled interventions in Sec. J further evaluate how the conditioning interface uses visual and task information. The design is also effective at a compact model scale: DriveVLA-W0 uses a 7B backbone and reports 90.2 PDMS on NAVSIM-v1 and 86.5 EPDMS on NAVSIM-v2, while our 0.5B model trained with SFT→GRPO achieves 92.2 and 90.6, respectively (Tables 1 and 2).

Meanwhile, stand-alone generative world models such as GAIA-1/GAIA-2 [11, 34] are primarily aimed at photorealistic driving scene synthesis and simulation/data augmentation/counterfactual scenario generation, so planning is not a direct task and the target of next-state prediction is closer to pixel-level video synthesis. In contrast, our next visual state prediction operates not in pixel space but in the visual token space on the LLM side, and does not aim to itself become a generative world model; its purpose is to supply the terminal condition the IK Network needs to recover trajectories.

In summary, while the GAIA family asks how the next scene may look, our predictor asks which visual state the ego should reach for the trajectory to be uniquely determined, and the two address different task formulations.

3 Motivations

3.1 Formulating of Existing Driving VLA Models

Existing Driving VLA planners are trained to directly regress trajectories. Let V_t denote the current visual observation (e.g., images or BEV tokens), E_t the ego status (velocity, yaw-rate, acceleration, etc.), C_t a text command (route-level intent), and $T_t \in \mathbb{R}^{H \times d}$ the H -step future trajectory (waypoints or controls). The training objective is then defined as:

$$\hat{T}_t = f_\theta(V_t, E_t, C_t), \quad \mathcal{L}_{\text{traj}}(\theta) = \mathbb{E} [\|T_t - f_\theta(V_t, E_t, C_t)\|_2^2], \quad (1)$$

or with text-based next token prediction

$$\hat{T}_t = f_\theta(V_t, E_t, C_t), \quad \mathcal{L}_{\text{traj}}(\theta) = \mathbb{E} \left[- \sum_{i=1}^L \log p_\theta(y_{t,i}^* \mid y_{t,<i}^*, V_t, E_t, C_t) \right]. \quad (2)$$

Directly regressing $\hat{T}_t = f_\theta(V_t, E_t, C_t)$ (Eq. (2)) requires the VLA model to obtain a feasible control sequence without an explicit boundary condition.

3.2 Phenomena due to Formulation of Existing VLA

Driving VLA Ignore Visual Informations. When deciding the next action in a driving situation, ego status, command, and visual information all serve as factors. To characterize the competition between ego-predictable behavior and vision-dependent corrections, we decompose the target trajectory into two regimes. In the majority regime ($\mathbb{I}_t = 0$), we assume that the short-horizon target is conditionally independent of vision given ego status and command; in the vision-critical regime, visual information supplies a trajectory correction. Sec. D gives the detailed derivation and empirical calibration.

$$T_t = T_0(E_t, C_t) + \mathbb{I}_t \cdot \Delta(V_t, E_t, C_t) + \xi_t, \quad (3)$$

where T_0 is the dominant behavior driven by ego status and command (e.g., lane-following, cruising, stopping), Δ is an action correction conditioned on visual features (e.g., obstacle avoidance, yield timing, intersection choice), and ξ_t is the noise inherent in the data. $\mathbb{I}_t \in \{0, 1\}$ indicates whether the current situation is vision-critical or not.

Substituting the target decomposition (Eq. 3) for T_t and letting $\varepsilon \triangleq \mathbb{P}(\mathbb{I}_t = 1)$, the loss function for trajectory prediction can be decomposed as follows:

$$\mathcal{L}(\theta) = (1 - \varepsilon) \mathcal{L}_0(\theta) + \varepsilon \mathcal{L}_1(\theta), \quad (4)$$

where $\mathcal{L}_0$ and $\mathcal{L}_1$ denote the conditional losses under $\mathbb{I}_t = 0$ and $\mathbb{I}_t = 1$, respectively.

For each (E, C) , we can write the predictor in non-vision-critical cases $\bar{f}_\theta(E, C)$ and the V -conditional residual $\tilde{f}_\theta(V, E, C)$ as:

$$\bar{f}_\theta(E, C) \triangleq \mathbb{E} [f_\theta(V, E, C) \mid E, C, \mathbb{I} = 0], \quad \tilde{f}_\theta(V, E, C) \triangleq f_\theta(V, E, C) - \bar{f}_\theta(E, C). \quad (5)$$

By definition, $\mathbb{E}[\tilde{f}_\theta(V, E, C) \mid E, C, \mathbb{I} = 0] = 0$. Under $(E, C, \mathbb{I} = 0)$, the target trajectory is $T_0(E, C) + \xi$ and does not depend on the visual feature V , so the conditional MSE under $\mathbb{I} = 0$ can be decomposed in bias–variance form as follows (see supp. material Sec. D.1 for the detailed derivation):

$$\begin{aligned} \mathcal{L}_0(\theta) &= \|T_0(E, C) - \bar{f}_\theta(E, C)\|_2^2 \\ &+ \mathbb{E} \left[\left\| \tilde{f}_\theta(V, E, C) \right\|_2^2 \mid E, C, \mathbb{I}=0 \right] + \mathbb{E} [\|\xi\|_2^2]. \end{aligned} \quad (6)$$

The first term is the bias corresponding to the average behavior that can be fit using only (E, C) , and the second term is the component whose output varies with V entering directly as a loss penalty in the majority regime. That is, the majority data weighted by $1 - \varepsilon$ pushes the learning signal toward $\tilde{f}_\theta \rightarrow 0$, causing the model to ignore visual features. Returning to the mixture loss (4), the learning signal that drives the model to use visual features comes only from the minority term weighted by ε , but in real driving data $\varepsilon \ll 1$, making it structurally difficult to learn vision-grounded planning. From this, we can interpret that shortcut learning through ego status and text command alone arises.

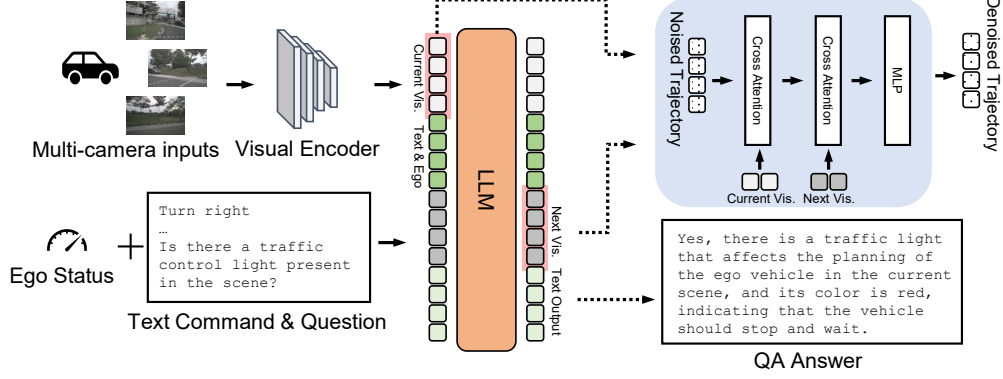


Figure 3: Overview of our model. The vision encoder extracts visual tokens V_t from the three front cameras; the LLM consumes (V_t, E, C) together with the QA prompt and is trained to predict the future visual state $\hat{V}_{t+\Delta}$ (and to answer driving-scene questions). The Inverse Kinematics Network, a cross-attention conditional diffusion model, decodes the trajectory from only the pair $(V_t, \hat{V}_{t+\Delta})$, without direct access to ego status E or text command C .

3.2.1 An Architectural Remedy without Additional Data.

We address the two shortcut pathways together: next-state prediction supplies dense visual supervision, and the IK Network removes direct ego-status and command access from trajectory decoding. Our key concept is to define the current visual feature as the current state and let the VLA predict the next state. Then, using the current/next states, trajectory prediction is performed through a separate inverse-kinematics module (IK module). Here, V_t denotes the 2D image feature obtained from the vision encoder. The LLM decoder w_ϕ of the VLA and the IK module h_ψ predict the next state $V_{t+\Delta}$ and the trajectory T_t , respectively. $V_{t+\Delta}$ serves as a proxy terminal condition and contributes to forming the IK solution of the VLA.

$$\hat{V}_{t+\Delta} = w_\phi(V_t, E_t, C_t), \quad \hat{T}_t = h_\psi(V_t, \hat{V}_{t+\Delta}). \quad (7)$$

To this end, the two objectives are jointly trained.

$$\mathcal{L}(\phi, \psi) = \lambda_{\text{state}} \mathbb{E} [\|V_{t+\Delta} - w_\phi(V_t, E_t, C_t)\|_2^2] + \lambda_{\text{traj}} \mathbb{E} [\|T_t - h_\psi(V_t, \hat{V}_{t+\Delta})\|_2^2]. \quad (8)$$

With this loss correction, concretely, the gradient with respect to θ_V is computed as:

$$\nabla_{\theta_V} \mathcal{L} = \lambda_{\text{state}} \nabla_{\theta_V} \mathcal{L}_{\text{state}} + \lambda_{\text{traj}} \nabla_{\theta_V} \mathcal{L}_{\text{traj}}, \quad (9)$$

where $\nabla_{\theta_V} \mathcal{L}_{\text{state}}$ does not vanish even in the $\mathbb{I}_t = 0$ situations that constitute most of the dataset, so that the visual parameters θ_V continue to receive a learning signal. The new loss obtained by this task formulation correction is intended to suppress shortcut learning induced by ego status and command.

- (i) Next state prediction provides dense supervision through visual features even in $\mathbb{I}_t = 0$ situations.
- (ii) The IK module h_ψ performs trajectory prediction using only the current/next state $(V_t, \hat{V}_{t+\Delta})$ information, without explicit ego status or command feature, which discourages the optimization from collapsing into a direct shortcut mapping $(E_t, C_t) \rightarrow T_t$.

4 Methods

4.1 Overview

Our VLA consists of three mechanisms: Next Visual Token Prediction, Question and Answering, and the Inverse Kinematics Network. We extract visual features by feeding front, front-left, and front-right images into a DINOv3 ViT-S/16 [39] encoder, and pass them through an MLP projector before inputting them to the LLM. The LLM also receives ego status and text instructions encoded through an MLP, as well as questions about the driving scene. As output, it predicts the next visual state at N seconds ahead, and, when given a question, outputs the answer as text. The current visual state and the next visual state are then fed into the Inverse Kinematics Network to perform trajectory prediction. The detailed architecture of our VLA is shown in Fig. 3.

4.2 Next Visual State Prediction

As its first task, our VLA performs ‘Next Visual State’ prediction. We extract visual features by feeding front, front-left, and front-right images into a DINOv3 ViT-S/16 [39] encoder, and pass them through an MLP projector before inputting them to the LLM. When fed into the LLM, the features are placed between `<scene_start>` and `<scene_end>`, and represented in text form by the placeholder `<SCENE>`. Unlike prior work [52], we do not directly emit a trajectory text output here, but instead let the model predict the next visual state at N seconds ahead. This prediction is trained to be formatted between `<next_start>` and `<next_end>` in the same format as the input. Concretely, we read out the LLM hidden states at the predicted `<next_start>` ... `<next_end>` positions in the assistant turn, and supervise them with an MSE loss against the ground-truth future visual tokens obtained by passing the next-frame images through the same vision encoder and projector. Writing H_t and A_{t-1} for the supplied trajectory history and previous-action context, respectively, the predictor and its visual supervision are

$$\hat{V}_{t+\Delta t} = w_\phi(V_t, E_t, C_t, H_t, A_{t-1}), \quad \mathcal{L}_{\text{visual}} = \text{MSE}(\hat{V}_{t+\Delta t}, V_{t+\Delta t}).$$

The LLM fuses visual evidence with ego kinematics and the textual route command: C_t specifies the intended maneuver, while E_t supplies motion context for predicting a compatible future state. The LLM encodes maneuver intent and ego kinematics in $\hat{V}_{t+\Delta t}$, allowing language to guide trajectory generation without direct text input to the IK decoder. See Appendix K for command interventions and representation probes.

4.3 Inverse Kinematics Network

As mentioned earlier, our method does not perform trajectory prediction directly within the LLM. We design an Inverse Kinematics Network (IK network), which takes only the current visual state seen earlier and the next visual state predicted by the LLM as input, and predicts the next trajectory. We stop trajectory-loss gradients at both conditioning inputs, separating visual-state learning from diffusion trajectory learning (Sec. E). Here, the constraint on the input is the key. The IK Network does not take ego status E or text command C as input, and is conditioned only on $(V_t, \hat{V}_{t+\Delta t})$. This design choice distinguishes our work from existing world-model + policy combinations (e.g., DriveVLA-W0 [23], LAW [22]), in which future prediction is used as an auxiliary signal while the trajectory decoder still has access to the full (E, C, V_t) . That is, our IK Network does not place the future visual state as an optional auxiliary input but uses the pair of two visual states as its primary condition, thereby suppressing the path along which an ego-status shortcut could otherwise flow.

As shown in Fig. 3, the IK network is constructed as a cross-attention-based conditional diffusion model. Specifically, we stack two cross-attention layers in which a noisy trajectory $x_t \in \mathbb{R}^{2 \times 6}$ serves as the query and the current visual state token $V_t \in \mathbb{R}^{N \times H}$ and the next visual state token $\hat{V}_{t+\Delta t} \in \mathbb{R}^{N \times H}$ predicted by the LLM serve as keys and values, predicting the noise ϵ . The timestep t is added to the query through a sinusoidal embedding. The denoising network ϵ_θ is then defined as:

$$\epsilon_\theta(x_t, t, V_t, \hat{V}_{t+\Delta t}) = \text{MLP}\left(\text{CrossAttn}(\text{CrossAttn}(x_t + \text{TE}(t), V_t), \hat{V}_{t+\Delta t})\right)$$

This model is trained with a DDPM [10] objective using the cosine β -schedule [32].

$$\mathcal{L}_{ik} = \mathbb{E}_{t, x_0, \epsilon} \left[\left\| \epsilon - \epsilon_\theta(\sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon, t, V_t, \hat{V}_{t+\Delta t}) \right\|_2^2 \right]$$

At inference, we start from $x_T \sim \mathcal{N}(0, I)$ and run 50 steps of DDPM sampling to obtain the final trajectory $T \in \mathbb{R}^{2 \times 6}$.

The IK Network turns the predicted visual state into an explicit input to action generation. Together with NSP supervision and gradient isolation, this interface couples trajectory decoding to a learned future scene while preserving the command and kinematic information needed to execute the maneuver.

4.4 Question and Answering

In parallel with the LLM’s next visual state prediction, we train question and answering using scene description datasets such as nuCaption [49] and NuX [6]. By conducting this jointly with next visual state prediction, we enable the LLM to perform question and answering about the driving scene while simultaneously improving its scene understanding ability. QA supervision strengthens the shared

representation used for future-state prediction; see Appendix R for the planning ablation. When a question is fed into the LLM, it is placed between `<question_start>` and `<question_end>`, and the answer is trained to appear between `<answer_start>` and `<answer_end>` in the output. This part is trained with next token prediction, identically to prior work [52].

$$\mathcal{L}(\theta) = -\frac{1}{T} \sum_{t=1}^T \log P_{\theta}(x_t | x_1, x_2, \dots, x_{t-1})$$

5 Experiments

5.1 Settings

Datasets and benchmarks. We train and evaluate our model on two types of autonomous driving benchmarks. NAVSIM [5] is a challenging closed-loop benchmark, and we evaluate on the NAVSIM-v1 and NAVSIM-v2 splits using PDMS and EPDMS, respectively, along with detailed metrics such as NC, DAC, and TTC. nuScenes [1] is a standard open-loop benchmark consisting of 700 (train) / 150 (val) scenes, where trajectory planning is evaluated by Average Displacement Error (ADE) and Collision Rate, and scene-level understanding is evaluated on nu-Caption [49], nu-X [6] and nuScenes-QA [33].

Architecture. We use Qwen2.5-0.5B-Instruct [48] as the LLM backbone and DINOv3 ViT-S/16 [39] as the visual encoder for both datasets, since no pretrained BEV encoder is publicly available for NAVSIM. The IK Network is a cross-attention-based conditional diffusion model trained with a cosine β -schedule [32] and run for 50 DDPM [10] sampling steps at inference. Details are provided in the supp. material (Sec. A).

Training Setup. On NAVSIM, we follow the 2-stage SFT $\rightarrow$ GRPO [37] procedure of RecogDrive [25] on the navtrain split, with a 4 s future frames and 8 output waypoints. On nuScenes, we train for 4 epochs jointly with nuScenes-QA, nu-Caption, and nu-X, using a 3 s frames and 6 waypoints. Details are listed in the supp. material (Sec. B).

Baselines. On nuScenes, we compare against recent Driving VLAs [17, 41, 44, 2], such as OpenDriveVLA [52]. On NAVSIM, in addition to a NAVSIM-adapted variant of OpenDriveVLA [52], we include VLA models spanning 2B–8B backbones [7, 23, 25, 30, 53, 4]. Since the original OpenDriveVLA relies on a UniAD BEV encoder trained on nuScenes but no pretrained BEV encoder corresponding to NAVSIM is publicly available, we reimplement OpenDriveVLA for NAVSIM (denoted *OpenDriveVLA** in the tables) by unifying the visual encoder with DINOv3 [39] as in our model and generating the trajectory as a direct text output of the LLM as in the original paper. This baseline shares with our model the visual encoder, LLM backbone (0.5B), training data, optimizer, learning rate schedule, warmup ratio, batch size, gradient clipping, encoder freeze policy, and the SFT $\rightarrow$ GRPO 2-stage training procedure used on NAVSIM, and the only difference between the two models is the introduction of the **next visual state prediction objective** and the **IK Network**. This matched comparison evaluates the combined NSP and IK design while controlling backbone capacity, data, and training schedule; the conditioning ablations further examine the individual contributions. Our model uses three perspective cameras without LiDAR or a BEV encoder, whereas DiffusionDrive and TransFuser use camera–LiDAR inputs [28, 3]. Sensor configurations are listed in the benchmark tables, with further comparison details in Appendix G.

5.2 Main Experiments

5.2.1 Experiments on NAVSIM.

As shown in Table 1, on NAVSIM-v1 our 0.5B model achieves the highest PDMS of 92.2, an improvement of +19.0 over the matched OpenDriveVLA* baseline (73.2) with the same backbone. This score exceeds those of the larger VLA baselines, including DriveVLA-W0 [23] (7B), ReCogDrive [25] (2B), AdaThinkDrive [30] (2B), and AutoVLA [53] (3B), and surpasses the 8B-scale DriveFine [4] (91.8) by 0.4 points. In terms of detailed metrics, our model also records No-Collision (NC) 99.1 and Time-To-Collision (TTC) 99.1, demonstrating strong performance on collision avoidance and time-to-collision in addition to overall PDMS.

As shown in Table 2, on the more challenging NAVSIM-v2 benchmark, our 0.5B model achieves the highest EPDMS of 90.6, exceeding the $16\times$ larger DriveFine-8B (89.7) by 0.9 points and DriveVLA-W0-7B (86.5) by 4.1 points. The gain over the matched OpenDriveVLA* baseline (70.2) is 20.4

Table 1: PDMS results on the NAVSIM-v1 benchmark. **Bold**=best, underline=second best. † denotes models trained with the SFT→GRPO schedule. OpenDriveVLA* shares all training hyperparameters, data, and the SFT→GRPO schedule with our model; the only differences are the next visual state prediction objective and the IK Network.

Method	Size	Sensors	NC↑	DAC↑	TTC↑	C.↑	EP↑	PDMS↑
Human	-	-	100	100	100	99.9	87.5	94.8
<i>End-to-End Methods</i>								
UniAD [15]	0.05B	C	97.8	91.9	92.9	100.0	78.8	83.4
TransFuser [3]	0.05B	C+L	97.7	92.8	92.8	100.0	79.2	84.0
LAW [22]	0.05B	C	96.4	95.4	88.7	99.9	81.7	84.6
Hydra-MDP [26]	0.05B	C+L	98.3	96.0	94.6	100.0	78.7	86.5
DiffusionDrive [28]	0.05B	C+L	98.2	96.2	94.7	100.0	82.2	88.1
WoTE [24]	0.05B	C+L	98.5	96.8	94.4	99.9	81.9	88.3
SUPER-AD [35]	0.05B	C	98.1	97.0	93.9	<u>99.8</u>	81.5	87.7
<i>Vision Language Action Methods</i>								
AutoVLA [53]	3B	C	98.4	95.6	<u>98.0</u>	99.9	81.9	89.1
DriveVLA-W0 [23]	7B	C	<u>98.7</u>	99.1	<u>95.3</u>	<u>99.3</u>	83.3	90.2
AdaThinkDrive [30]	2B	C	98.4	97.8	95.2	100	84.4	90.3
ReCogDrive [25]	2B	C	98.2	97.8	95.2	99.8	83.5	89.6
DriveFine [4]	8B	C	98.8	<u>98.6</u>	96.2	100	86.9	<u>91.8</u>
OpenDriveVLA* [52]	0.5B	C	92.2	83.2	91.2	99.9	68.7	73.2
Ours (SFT)	0.5B	C	99.1 (+6.9)	96.2(+13.0)	<u>99.0</u> (+7.8)	100.0 (+0.1)	83.4(+14.7)	90.3(+17.1)
Ours†	0.5B	C	99.1 (+6.9)	97.8(+14.6)	99.1 (+7.9)	<u>99.9</u> (+0.0)	<u>85.4</u> (+16.7)	92.2 (+19.0)

Sensors: C = cameras; L = LiDAR; C+L = cameras and LiDAR. Entries describe inference-time scene sensors, not ego status or training supervision.

Table 2: EPDMS results on the NAVSIM-v2 benchmark. **Bold**=best, underline=second best.

Method	Size	Sensors	NC↑	DAC↑	DDC↑	TLC↑	EP↑	TTC↑	LK↑	C.↑	EC↑	EPDMS↑
Ego-MLP [27]	0.05B	-	93.1	77.9	92.7	99.6	86.0	91.5	89.4	<u>98.3</u>	85.4	64.0
TransFuser [3]	0.05B	C+L	96.9	89.9	97.8	99.7	87.1	95.4	92.7	<u>98.3</u>	87.2	76.7
DriveSuprim [50]	0.05B	C	97.5	96.5	99.4	99.6	<u>88.4</u>	96.6	95.5	<u>98.3</u>	77.0	83.1
ARTEMIS [7]	-	C+L	98.3	95.1	98.6	99.8	81.5	97.4	96.5	<u>98.3</u>	-	83.1
ReCogDrive [25]	2B	C	98.3	95.2	99.5	<u>99.8</u>	87.1	97.5	96.6	<u>98.3</u>	86.5	83.6
DiffusionDrive [28]	0.05B	C+L	98.2	95.9	99.4	<u>99.8</u>	87.5	97.3	96.8	<u>98.3</u>	87.7	84.5
SUPER-AD [35]	0.05B	C	98.1	97.3	98.7	<u>99.8</u>	86.8	97.3	97.5	<u>98.3</u>	86.1	84.3
DriveVLA-W0 [23]	7B	C	99.0	98.4	99.3	99.9	87.0	98.1	93.2	<u>97.9</u>	58.9	86.5
DriveFine [4]	8B	C	98.7	97.3	99.5	<u>99.8</u>	88.7	97.8	97.7	98.4	83.8	89.7
OpenDriveVLA* [52]	0.5B	C	92.2	83.2	93.8	99.4	89.3	91.2	90.8	98.3	76.8	70.2
Ours (SFT)	0.5B	C	<u>99.1</u> (+6.9)	96.2(+13.0)	99.5 (+5.7)	99.9 (+0.5)	87.7(-1.6)	<u>99.1</u> (+7.9)	<u>97.5</u> (+6.7)	98.2(-0.1)	84.4(+7.6)	89.3(+19.1)
Ours†	0.5B	C	99.2 (+7.0)	<u>97.8</u> (+14.6)	99.5 (+5.7)	99.9 (+0.5)	<u>88.4</u> (-0.9)	99.2 (+8.0)	97.4(+6.6)	98.2(-0.1)	81.3(+4.5)	90.6 (+20.4)

Sensors: C = cameras; L = LiDAR; C+L = cameras and LiDAR; - = no scene-sensor input for Ego-MLP. Sensors describe inference inputs, not training supervision. ARTEMIS uses a ResNet-34-based sensor encoder; its parameter count is left unspecified. †: SFT→GRPO.

points. Our model also achieves the highest NC (99.2) and TTC (99.2), and ties the highest TLC (99.9), demonstrating strong collision avoidance and traffic-light compliance. Lane Keeping reaches 97.4.

These results have two implications. First, since the only difference from the OpenDriveVLA* baseline is our next visual state prediction and IK Network, the +19.0 PDMS and +20.4 EPDMS gains support the combined design under the matched configuration. Second, the fact that a 0.5B light model has reached driving performance on par with 7B–8B VLAs suggests that structural changes that force active use of visual features may have a greater impact on trajectory planning accuracy than simply scaling up the LLM.

5.2.2 Experiments on nuScenes.

As shown in Table 3, our 0.5B model achieves an ADE of 0.06, reducing the error by −0.27 relative to the 14× larger 7B-scale OpenDriveVLA, and also records a collision rate of 0.09 at a level equal to or above it. Together with the component ablations below, these results show that the visual grounding introduced by next-state prediction and the IK Network translates into more accurate trajectory planning, enabling our 0.5B model to outperform the 7B-scale OpenDriveVLA with a

Table 3: Open-Loop planning performance on nuScenes *val* set. “-” indicates the metric is not reported for that method in the paper’s tables. **Bold**=best, Underline=second.

Method	ST-P3 metrics								UniAD metrics								LLM	Input	Sensors	
	L2 (m) ↓				Collision (%) ↓				L2 (m) ↓				Collision (%) ↓							
	1s	2s	3s	Avg.	1s	2s	3s	Avg.	1s	2s	3s	Avg.	1s	2s	3s	Avg.				
None-Autoregressive Methods																				
ST-P3 [13]	1.33	2.11	2.90	2.11	0.23	0.62	1.27	0.71	-	-	-	-	-	-	-	-	-	Visual	C	
VAD [19]	0.17	0.34	0.60	0.37	0.07	0.10	0.24	0.14	-	-	-	-	-	-	-	-	-	Visual	C	
Ego-MLP [51]	0.46	0.76	1.12	0.78	0.21	0.35	0.58	0.38	-	-	-	-	-	-	-	-	-	Ego	-	
UniAD [15]	0.44	0.67	0.96	0.69	0.04	0.08	0.23	0.12	0.48	0.96	1.65	1.03	0.05	0.17	0.71	0.31	-	Visual	C	
InsightDrive [40]	0.23	0.41	0.68	0.44	0.09	0.10	0.27	0.15	0.30	0.72	1.41	0.81	0.08	0.15	0.84	0.36	-	Visual	C	
FF [12]	-	-	-	-	-	-	-	-	0.55	1.20	2.54	1.43	0.06	0.17	1.07	0.43	-	LiDAR	L	
EO [20]	-	-	-	-	-	-	-	-	0.67	1.36	2.78	1.60	0.04	0.09	0.88	0.33	-	LiDAR	L	
Autoregressive Methods																				
GPVL [21]	0.21	0.39	0.69	0.43	0.07	0.09	0.27	0.14	-	-	-	-	-	-	-	-	-	BERT	Textual	C
DriveVLM [41]	0.18	0.34	0.68	0.40	0.10	0.22	0.45	0.27	-	-	-	-	-	-	-	-	-	Qwen-VL-7B	Visual	C
GPT-Driver [31]	0.20	0.40	0.70	0.44	0.04	0.12	0.36	0.17	0.27	0.74	1.52	0.84	0.07	0.15	1.10	0.44	-	GPT-3.5	Textual	-
RDA-Driver [16]	0.17	0.37	0.69	0.40	0.01	0.05	0.26	0.10	0.23	0.73	1.54	0.80	0.00	0.13	0.83	0.32	-	LLaVa-7B	Visual	C
OminiDrive [42]	0.14	0.29	0.55	0.33	0.00	0.13	0.78	0.30	-	-	-	-	-	-	-	-	-	LLaVa-7B	Visual	C
EMMA [17]	0.14	0.29	0.54	0.32	-	-	-	-	-	-	-	-	-	-	-	-	-	Gemini	Visual	C
OpenEMMA [45]	1.45	3.21	3.76	2.81	-	-	-	-	-	-	-	-	-	-	-	-	-	Qwen-VL-7B	Visual	C
DME-Driver [9]	-	-	-	-	-	-	-	-	0.45	0.91	1.58	0.98	0.05	0.28	0.55	0.29	-	LLaVa-7B	Visual	C
OpenDriveVLA-0.5B [52]	0.15	0.32	0.57	0.35	0.01	0.06	0.20	0.09	0.21	0.60	1.22	0.68	0.00	0.15	0.63	0.26	-	Qwen2.5-0.5B	Visual	C
OpenDriveVLA-3B [52]	0.14	0.30	0.55	0.33	0.02	0.07	0.22	0.10	0.19	0.58	1.24	0.67	0.02	0.18	0.70	0.30	-	Qwen2.5-3B	Visual	C
OpenDriveVLA-7B [52]	0.15	0.31	0.55	0.33	0.01	0.08	0.21	0.10	0.20	0.58	1.21	0.66	0.00	0.22	0.55	0.25	-	Qwen2.5-7B	Visual	C
Impromptu-VLA-3B [2]	0.13	0.27	0.52	0.30	-	-	-	-	-	-	-	-	-	-	-	-	-	Qwen2.5-VL-3B	Visual	C
Impromptu-VLA-7B [2]	0.13	0.27	0.53	0.30	-	-	-	-	-	-	-	-	-	-	-	-	-	Qwen2.5-VL-7B	Visual	C
Ours-0.5B	0.04	0.06	0.10	0.06	0.07	0.08	0.11	0.09	0.05	0.08	0.25	0.13	0.02	0.03	0.12	0.06	-	Qwen2.5-0.5B	Visual	C

Sensors: C = cameras; L = LiDAR; C+L = cameras and LiDAR. Sensors denote the scene-sensing modality, whereas Input denotes the planner’s representation. Ego-MLP uses ego status only. GPT-Driver consumes textual perception/prediction outputs; - leaves the upstream sensor/annotation configuration of the quoted result unspecified. Camera-based BEV encoders (e.g., OpenDriveVLA’s UniAD encoder) are labeled C, not C+L. Ours uses perspective-camera features without a BEV encoder.

Table 4: Obstacle stitching experiment.

Model	Decel. Ratio	Avg. Δ displ
Impromptu-VLA 3B [2]	38.8%	-0.00m
Alpamayo-R1 10B [44]	54.5%	-0.12m
OpenDriveVLA 0.5B [52]	44.2%	-0.06m
Ours 0.5B	63.8%	-1.04m

Table 5: Ablation study on nuScenes *val*.

Method	ADE (m)↓
w/o Next State Prediction (current state pred.)	0.07
w/o IK Network (direct MLP traj. pred.)	0.31
w/o Diffusion in IK (cross-attn pooling + MLP)	0.16
Ours	0.06

14× smaller backbone. As a side effect of the recovered visual grounding, our model also shows consistent improvements over OpenDriveVLA on the nu-Caption captioning and nu-X reasoning benchmarks. Detailed results are reported in the appendix (Secs. S.1 to S.3).

5.3 Analysis

5.3.1 Counterfactual Stitching.

We first revisit the counterfactual probe introduced as the motivation in Fig. 1 of Sec. 1, where a near-range vehicle sprite is synthesized in front of the ego on nuScenes *val* while all textual inputs (ego status, command, history) are held fixed. Visually, our model is the only one that bends or shortens the trajectory in response to the stitched obstacle, while the baselines maintain a near-identical path before and after the perturbation. Table 4 quantifies this gap: our 0.5B model decelerates on 63.8% of perturbed samples with an average 3-second endpoint shortening of −1.04 m, far exceeding OpenDriveVLA-0.5B (44.2%, −0.06 m) and even the much larger Alpamayo-R1-10B (54.5%, −0.12 m). This suggests that, unlike existing Driving VLAs, our model relies on visual features when shaping the trajectory rather than treating them as a largely unused side input. Furthermore, in a paired control where the sprite scale is held fixed and only its position is changed (road horizon vs. sky), 65.5% of samples show a larger response when the sprite lies on the road (signed paired mean −0.64 m), indicating that the response is genuinely position-aware rather than a generic size-based reaction; full results are in the supp. material (Sec. Q).

Complementary GradCAM results and their measurement protocol are provided in Appendix P.

5.3.2 Ablation Study

The stitching results suggest that visual features play a meaningful role in shaping our trajectory output. We finally ask which architectural choices contribute to this behavior; Table 5 summarizes the contribution of each component on nuScenes. Replacing the IK Network with a small MLP head on the LLM’s output hidden state degrades ADE from 0.06 to 0.31 ($\sim 5\times$), suggesting that the input bottleneck of the trajectory decoder plays a substantial role in maintaining visual grounding. Replacing next state prediction with current state reconstruction yields a small average drop ($0.06 \rightarrow 0.07$), but the gap concentrates in dynamic regimes such as turning ($p = 1.2 \times 10^{-14}$ on ego angular velocity, see supp. material Sec. H). Replacing the diffusion-based IK with a deterministic cross-attention pooling head gives ADE 0.16, still far better than removing the IK Network and indicating that generative trajectory decoding offers a further refinement on top of the visual-only bottleneck.

6 Conclusion

In this paper, we have addressed *blind planning* of existing Driving VLAs. We pointed out that, when trajectory generation is viewed as an inverse kinematics problem, existing VLAs lack a future visual state to anchor the trajectory, and we proposed a next visual state prediction objective that supplies this signal directly as supervision for the LLM, together with an Inverse Kinematics Network that decodes trajectories primarily from the current and future visual states, suppressing reliance on ego-status and text shortcuts. Counterfactual interventions demonstrate improved hazard-specific visual responsiveness, while controlled ablations establish the contributions of the state-conditioning interface, next-state prediction, gradient isolation, and QA co-training. Our camera-only 0.5B model achieves state-of-the-art planning performance on NAVSIM-v1, NAVSIM-v2, and nuScenes. These results establish visual-state-conditioned action decoding as an effective design for grounding driving VLAs.

Acknowledgements.

References

- [1] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom. nusenes: A multimodal dataset for autonomous driving. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11621–11631, 2020.
- [2] H. Chi, H.-a. Gao, Z. Liu, J. Liu, C. Liu, J. Li, K. Yang, Y. Yu, Z. Wang, W. Li, et al. Impromptu vla: Open weights and open data for driving vision-language-action models. *arXiv preprint arXiv:2505.23757*, 2025.
- [3] K. Chitta, A. Prakash, B. Jaeger, Z. Yu, K. Renz, and A. Geiger. Transfuser: Imitation with transformer-based sensor fusion for autonomous driving. *IEEE transactions on pattern analysis and machine intelligence*, 45(11):12878–12895, 2022.
- [4] C. Dang, S. Ang, Y. Li, H. Tian, J. Wang, G. Li, H. Ye, J. Ma, L. Chen, and Y. Wang. Drivefine: Refining-augmented masked diffusion vla for precise and robust driving. *arXiv preprint arXiv:2602.14577*, 2026.
- [5] D. Dauner, M. Hallgarten, T. Li, X. Weng, Z. Huang, Z. Yang, H. Li, I. Gilitschenski, B. Ivanovic, M. Pavone, A. Geiger, and K. Chitta. NAVSIM: Data-driven non-reactive autonomous vehicle simulation and benchmarking. In *Advances in Neural Information Processing Systems*, 2024.
- [6] K. Ding, B. Chen, Y. Su, H.-a. Gao, B. Jin, C. Sima, W. Zhang, X. Li, P. Barsch, H. Li, et al. Hint-ad: Holistically aligned interpretability in end-to-end autonomous driving. *arXiv preprint arXiv:2409.06702*, 2024.
- [7] R. Feng, N. Xi, D. Chu, R. Wang, Z. Deng, A. Wang, L. Lu, J. Wang, and Y. Huang. Artemis: Autoregressive end-to-end trajectory planning with mixture of experts for autonomous driving. *IEEE Robotics and Automation Letters*, 11(1):226–233, 2025.
- [8] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

- [9] W. Han, D. Guo, C.-Z. Xu, and J. Shen. Dme-driver: Integrating human decision logic and 3d scene perception in autonomous driving. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 3347–3355, 2025.
- [10] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
- [11] A. Hu, L. Russell, H. Yeo, Z. Murez, G. Fedoseev, A. Kendall, J. Shotton, and G. Corrado. Gaia-1: A generative world model for autonomous driving. *arXiv preprint arXiv:2309.17080*, 2023.
- [12] P. Hu, A. Huang, J. Dolan, D. Held, and D. Ramanan. Safe local motion planning with self-supervised freespace forecasting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12732–12741, 2021.
- [13] S. Hu, L. Chen, P. Wu, H. Li, J. Yan, and D. Tao. St-p3: End-to-end vision-based autonomous driving via spatial-temporal feature learning. In *European Conference on Computer Vision*, pages 533–549. Springer, 2022.
- [14] T. Hu, X. Liu, S. Wang, Y. Zhu, A. Liang, L. Kong, G. Zhao, Z. Gong, J. Cen, Z. Huang, et al. Vision-language-action models for autonomous driving: Past, present, and future. *arXiv preprint arXiv:2512.16760*, 2025.
- [15] Y. Hu, J. Yang, L. Chen, K. Li, C. Sima, X. Zhu, S. Chai, S. Du, T. Lin, W. Wang, et al. Planning-oriented autonomous driving. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 17853–17862, 2023.
- [16] Z. Huang, T. Tang, S. Chen, S. Lin, Z. Jie, L. Ma, G. Wang, and X. Liang. Making large language models better planners with reasoning-decision alignment. In *European Conference on Computer Vision*, pages 73–90. Springer, 2024.
- [17] J.-J. Hwang, R. Xu, H. Lin, W.-C. Hung, J. Ji, K. Choi, D. Huang, T. He, P. Covington, B. Sapp, et al. Emma: End-to-end multimodal model for autonomous driving. *arXiv preprint arXiv:2410.23262*, 2024.
- [18] X. Jia, Z. Yang, Q. Li, Z. Zhang, and J. Yan. Bench2drive: Towards multi-ability benchmarking of closed-loop end-to-end autonomous driving. *Advances in Neural Information Processing Systems*, 37:819–844, 2024.
- [19] B. Jiang, S. Chen, Q. Xu, B. Liao, J. Chen, H. Zhou, Q. Zhang, W. Liu, C. Huang, and X. Wang. Vad: Vectorized scene representation for efficient autonomous driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 8340–8350, 2023.
- [20] T. Khurana, P. Hu, A. Dave, J. Ziglar, D. Held, and D. Ramanan. Differentiable raycasting for self-supervised occupancy forecasting. In *European Conference on Computer Vision*, pages 353–369. Springer, 2022.
- [21] T. Li, H. Wang, X. Li, W. Liao, T. He, and P. Peng. Generative planning with 3d-vision language pre-training for end-to-end autonomous driving. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 4950–4958, 2025.
- [22] Y. Li, L. Fan, J. He, Y. Wang, Y. Chen, Z. Zhang, and T. Tan. Enhancing end-to-end autonomous driving with latent world model. *arXiv preprint arXiv:2406.08481*, 2024.
- [23] Y. Li, S. Shang, W. Liu, B. Zhan, H. Wang, Y. Wang, Y. Chen, X. Wang, Y. An, C. Tang, et al. Drivevla-w0: World models amplify data scaling law in autonomous driving. *arXiv preprint arXiv:2510.12796*, 2025.
- [24] Y. Li, Y. Wang, Y. Liu, J. He, L. Fan, and Z. Zhang. End-to-end driving with online trajectory evaluation via bev world model. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 27137–27146, 2025.
- [25] Y. Li, K. Xiong, X. Guo, F. Li, S. Yan, G. Xu, L. Zhou, L. Chen, H. Sun, B. Wang, et al. Recogdrive: A reinforced cognitive framework for end-to-end autonomous driving. *arXiv preprint arXiv:2506.08052*, 2025.

- [26] Z. Li, K. Li, S. Wang, S. Lan, Z. Yu, Y. Ji, Z. Li, Z. Zhu, J. Kautz, Z. Wu, et al. Hydra-mdp: End-to-end multimodal planning with multi-target hydra-distillation. *arXiv preprint arXiv:2406.06978*, 2024.
- [27] Z. Li, Z. Yu, S. Lan, J. Li, J. Kautz, T. Lu, and J. M. Alvarez. Is ego status all you need for open-loop end-to-end autonomous driving? In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14864–14873, 2024.
- [28] B. Liao, S. Chen, H. Yin, B. Jiang, C. Wang, S. Yan, X. Zhang, X. Li, Y. Zhang, Q. Zhang, et al. Diffusiondrive: Truncated diffusion model for end-to-end autonomous driving. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 12037–12047, 2025.
- [29] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- [30] Y. Luo, F. Li, S. Xu, Z. Lai, L. Yang, Q. Chen, Z. Luo, Z. Xie, S. Jiang, J. Liu, et al. Ada-thinkdrive: Adaptive thinking via reinforcement learning for autonomous driving. *arXiv preprint arXiv:2509.13769*, 2025.
- [31] J. Mao, Y. Qian, J. Ye, H. Zhao, and Y. Wang. Gpt-driver: Learning to drive with gpt. *arXiv preprint arXiv:2310.01415*, 2023.
- [32] A. Q. Nichol and P. Dhariwal. Improved denoising diffusion probabilistic models. In *International conference on machine learning*, pages 8162–8171. PMLR, 2021.
- [33] T. Qian, J. Chen, L. Zhuo, Y. Jiao, and Y.-G. Jiang. Nuscenes-qa: A multi-modal visual question answering benchmark for autonomous driving scenario. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 4542–4550, 2024.
- [34] L. Russell, A. Hu, L. Bertoni, G. Fedoseev, J. Shotton, E. Arani, and G. Corrado. Gaia-2: A controllable multi-view generative world model for autonomous driving. *arXiv preprint arXiv:2503.20523*, 2025.
- [35] W. Ryu, S. Yu, S. Moon, H. Choi, J. Park, J. Kim, and H. Shim. SUPER-AD: Semantic uncertainty-aware planning for end-to-end robust autonomous driving, 2025. URL <https://arxiv.org/abs/2511.22865>.
- [36] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra. Grad-cam: visual explanations from deep networks via gradient-based localization. *International journal of computer vision*, 128(2):336–359, 2020.
- [37] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [38] C. Sima, K. Renz, K. Chitta, L. Chen, H. Zhang, C. Xie, J. Beißwenger, P. Luo, A. Geiger, and H. Li. Drivelm: Driving with graph visual question answering. In *European conference on computer vision*, pages 256–274. Springer, 2024.
- [39] O. Siméoni, H. V. Vo, M. Seitzer, F. Baldassarre, M. Oquab, C. Jose, V. Khalidov, M. Szafraniec, S. Yi, M. Ramamonjisoa, et al. Dinov3. *arXiv preprint arXiv:2508.10104*, 2025.
- [40] R. Song, X. Guo, Y. Peng, Q. Wei, H. Wu, and L. Chen. Insightdrive: Insight scene representation for end-to-end autonomous driving. *arXiv preprint arXiv:2503.13047*, 2025.
- [41] X. Tian, J. Gu, B. Li, Y. Liu, Y. Wang, Z. Zhao, K. Zhan, P. Jia, X. Lang, and H. Zhao. Drivelm: The convergence of autonomous driving and large vision-language models. *arXiv preprint arXiv:2402.12289*, 2024.
- [42] S. Wang, Z. Yu, X. Jiang, S. Lan, M. Shi, N. Chang, J. Kautz, Y. Li, and J. M. Alvarez. Omnidrive: A holistic vision-language dataset for autonomous driving with counterfactual reasoning. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 22442–22452, 2025.

- [43] Y. Wang, J. He, L. Fan, H. Li, Y. Chen, and Z. Zhang. Driving into the future: Multiview visual forecasting and planning with world model for autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14749–14759, 2024.
- [44] Y. Wang, W. Luo, J. Bai, Y. Cao, T. Che, K. Chen, Y. Chen, J. Diamond, Y. Ding, W. Ding, et al. Alpamayo-r1: Bridging reasoning and action prediction for generalizable autonomous driving in the long tail. *arXiv preprint arXiv:2511.00088*, 2025.
- [45] S. Xing, C. Qian, Y. Wang, H. Hua, K. Tian, Y. Zhou, and Z. Tu. Openemima: Open-source multimodal model for end-to-end autonomous driving. In *Proceedings of the Winter Conference on Applications of Computer Vision*, pages 1001–1009, 2025.
- [46] Z. Xing, X. Zhang, Y. Hu, B. Jiang, T. He, Q. Zhang, X. Long, and W. Yin. Goalflow: Goal-driven flow matching for multimodal trajectories generation in end-to-end autonomous driving. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 1602–1611, 2025.
- [47] Z. Xu, Y. Zhang, E. Xie, Z. Zhao, Y. Guo, K.-Y. K. Wong, Z. Li, and H. Zhao. Drivegpt4: Interpretable end-to-end autonomous driving via large language model. *IEEE Robotics and Automation Letters*, 2024.
- [48] A. Yang et al. Qwen2.5 technical report, 2024. URL <https://arxiv.org/abs/2412.15115>.
- [49] S. Yang, J. Liu, R. Zhang, M. Pan, Z. Guo, X. Li, Z. Chen, P. Gao, H. Li, Y. Guo, et al. Lidar-llm: Exploring the potential of large language models for 3d lidar understanding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 9247–9255, 2025.
- [50] W. Yao, Z. Li, S. Lan, Z. Wang, X. Sun, J. M. Alvarez, and Z. Wu. Drivesuprim: Towards precise trajectory selection for end-to-end planning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 11910–11918, 2026.
- [51] J.-T. Zhai, Z. Feng, J. Du, Y. Mao, J.-J. Liu, Z. Tan, Y. Zhang, X. Ye, and J. Wang. Rethinking the open-loop evaluation of end-to-end autonomous driving in nuscen. *arXiv preprint arXiv:2305.10430*, 2023.
- [52] X. Zhou, X. Han, F. Yang, Y. Ma, V. Tresp, and A. Knoll. Opendrivevla: Towards end-to-end autonomous driving with large vision language action model. *arXiv preprint arXiv:2503.23463*, 2025.
- [53] Z. Zhou, T. Cai, S. Z. Zhao, Y. Zhang, Z. Huang, B. Zhou, and J. Ma. Autovla: A vision-language-action model for end-to-end autonomous driving with adaptive reasoning and reinforcement fine-tuning. *arXiv preprint arXiv:2506.13757*, 2025.
- [54] J. Zou, S. Chen, B. Liao, Z. Zheng, Y. Song, L. Zhang, Q. Zhang, W. Liu, and X. Wang. Diffusiondrivev2: Reinforcement learning-constrained truncated diffusion modeling in end-to-end autonomous driving. *arXiv preprint arXiv:2512.07745*, 2025.

A Architecture Details

Visual encoder and LLM input. We extract patch-level visual tokens with DINOv3 ViT-S/16 [39] from the three front cameras (front, front-left, front-right), pass them through an MLP projector, and inject the resulting tokens into the LLM (Qwen2.5-0.5B-Instruct [48]). On the input side, the visual tokens are placed between the special tokens `<scene_start>` and `<scene_end>`, and represented in text form by the placeholder `<SCENE>`. The LLM additionally receives the ego status and text instruction encoded through an MLP, and (when present) a question about the driving scene wrapped between `<question_start>` and `<question_end>`.

LLM output. The LLM is trained to emit the visual state of the future scene between `<scene_start>` and `<scene_end>` in its output, and the next visual state $\hat{V}_{t+\Delta}$ is read out from this region. For QA, the answer appears between `<answer_start>` and `<answer_end>` and is supervised with next-token prediction.

Inverse Kinematics Network. The IK Network is a cross-attention-based conditional diffusion model with hidden dimension $d_{\text{model}} = 256$, 8 attention heads, and 2 cross-attention blocks. The noisy trajectory serves as the query, and the current visual state tokens V_t and the predicted next visual state tokens $\hat{V}_{t+\Delta}$ serve as keys/values; the timestep is added to the query through a sinusoidal embedding. Diffusion training uses the cosine β -schedule [32] and at inference we run 50 DDPM [10] sampling steps starting from $x_T \sim \mathcal{N}(0, I)$.

B Training Hyperparameters

Optimization. We use AdamW [29] with a cosine learning-rate schedule and linear warmup (warmup ratio 0.145) and gradient clipping 1.0 on both datasets. Considering the difference in capacity and training stability between the LLM backbone and the IK Network, we apply different learning rates of 1×10^{-5} and 1×10^{-4} , respectively.

NAVSIM. We follow the 2-stage training procedure proposed by RecogDrive [25]: (i) supervised fine-tuning (SFT) on the navtrain split, followed by (ii) GRPO [37] reinforcement learning on the same split with PDMS-based reward signals. The future prediction horizon is 4 s and the IK Network outputs 8 waypoints.

nuScenes. The main-table model is trained on the nuScenes train split for 4 epochs and additionally on the language dataset for 1 epoch. The controlled conditioning ablation in Sec. I omits this additional language-dataset epoch to isolate the architecture and conditioning effects from that extra language training. The trajectory planning loss and the next visual state prediction loss are combined with weight 1.0 each, and nuScenes-QA, nu-Caption, and nu-X are jointly trained as multi-task supervision through the same LLM. The future prediction horizon is 3 s and the IK Network outputs 6 waypoints.

C Input Prompt Format

We follow a Qwen-style ChatML conversation template with two roles, `system` and `user`, and condition the LLM on a structured prompt rather than free-form text. This section makes the exact format explicit.

Special tokens. On top of the base Qwen2.5 vocabulary we add the following *additional special tokens*, each encoded as a single token: `<scene_start>`, `<scene_end>`, `<SCENE>` (placeholder for the visual tokens V_t); `<ego_start>`, `<ego_end>`, `<EGO>` (placeholder for the projected ego-status feature); `<next_start>`, `<next_end>`, `<NEXT_STATE>` (assistant-side placeholders into which $\hat{V}_{t+\Delta}$ is read out); `<question_start>`, `<question_end>`, `<answer_start>`, `<answer_end>` for QA wrappers. The visual placeholders `<SCENE>`, `<EGO>`, and `<VISUAL_STATE>` are replaced by the corresponding feature embeddings at the input/output embedding stage rather than by tokenized text.

System prompt. The system message specifies the role, the locations of the visual and ego placeholders, the coordinate convention, the two tasks (next visual state prediction and optional QA), and the strict output format:

```
You are DriveIK, a vision-language driving world model with latent
actions.
You are given:
- Current visual state tokens:
<scene_start><SCENE><scene_end>
- Ego status: <ego_start><EGO><ego_end>
You will be given:
- Previous action trajectory.
- Optional nuScenes QA question.
Coordinate convention (IMPORTANT):
- Ego-centric 2D BEV coordinates in meters; x is to the right, y is
to the front.
Your tasks:
1) Predict the NEXT visual state (one step ahead). To predict it,
you MUST internally infer a feasible ego trajectory; DO NOT output
the trajectory -- it is handled by a separate inverse-kinematics
```

```

model.
2) If a QA question is provided, output the QA answer.
Output format (STRICT). Output ONLY the blocks below.
Block 1 (next visual state): <visual_start> <VISUAL_STATE> ...
<VISUAL_STATE> <visual_end>
Block 2 (optional QA): <answer_start>SHORT_ANSWER <answer_end>

```

The number of <VISUAL_STATE> placeholders matches the spatial token count of $\hat{V}_{t+\Delta}$, and the LLM hidden states at those positions are read out as the predicted next visual state.

User prompt. The user message follows an OpenDriveVLA-style structured layout that we build from the per-sample driving meta-data. Concretely, each user prompt contains the lines below:

```

Ego status: <ego-status string>
Historical trajectory (last 2 seconds): [(x1,y1),(x2,y2),(x3,y3),(x4,y4)]
Mission goal: {turn right | turn left | keep forward}
Previous action: <previous action string>
<question_start><QA question> <question_end> (optional)

```

The ego-status string is built from `gt_ego_lcf_feat` and `gt_ego_his_diff` fields of the nuScenes info dict and has the form “- Velocity (vx,vy): (·,·) - Heading Angular Velocity (v_yaw): (·) - Acceleration (ax,ay): (·,·) - Can Bus: (·,·) - Heading Speed: (·) - Steering: (·)”. The historical trajectory line is filled with 4 recent ego positions in the same $(x_{\text{right}}, y_{\text{front}})$ convention used by the system prompt. The mission goal is a one-of-three textual command derived from `gt_ego_fut_cmd`.

Assistant output. The assistant turn produces, in order, (i) the visual state block <next_start> <NEXT_STATE>^K, <next_end>, where K matches the spatial token count of $\hat{V}_{t+\Delta}$, and (ii) optionally the answer block <answer_start> <SHORT_ANSWER>, <answer_end> when a QA question was provided. Trajectories are *not* produced by the LLM; the IK Network consumes the hidden states at the <NEXT_STATE> positions, together with the current visual tokens, and outputs the trajectory through cross-attention diffusion.

D Detailed Analysis for Section Sec. 3

This section derives the loss mechanism behind ego-status shortcut learning and explains how next visual state prediction and the IK Network address it. The analysis identifies an ego-only optimum in the majority regime and the prediction error incurred by ignoring visual information under future-state supervision. Ego-status extrapolation calibrates the regime imbalance, while counterfactual stitching and controlled ablations establish the behavioral effects of our design.

The ego-predictable majority regime. For the short-horizon target trajectory T_t (3–4 s in our experiments), we assume

$$T_t \perp V_t \mid (E, C, \mathbb{I}_t = 0), \quad T_t = T_0(E, C) + \xi_t \quad \text{when } \mathbb{I}_t = 0, \quad (10)$$

with zero-mean noise independent of the inputs within this regime. This assumption formalizes short-horizon cases in which the recorded trajectory is predictable from ego status and command, with little additional information supplied by vision. Visual constraints already reflected in speed, yaw rate, and command make such trajectories readily extrapolated from (E, C) . The vision-critical regime ($\mathbb{I}_t = 1$) comprises cases where lane markings, traffic lights, speed limits, road geometry, or other visual evidence provide additional information that changes the target trajectory conditioned on (E, C) . Thus, the indicator captures the incremental predictive role of vision across maneuver categories. The exact decomposition below uses the conditional-independence idealization in Eq. (10).

Empirical calibration of the vision-critical minority. Vision-critical examples constitute only a small minority under our ego-status-gap proxy: approximately 10–30%, compared with a 70–90% majority with smaller kinematic prediction errors. Motivated by prior analyses of ego-status shortcuts [27, 51], Sec. D.4 quantifies this imbalance by extrapolating trajectories from current speed and yaw rate without visual input. On nuScenes validation, the median ego-status gap is

1.286 m; only 30.0% of samples exceed 2.14 m, and just 9.8% exceed 5 m (Table 6). Thus, across these proxy thresholds, examples with large errors under ego-only extrapolation are consistently outnumbered by those with smaller errors. This imbalance supplies far more training examples for learning an (E, C) shortcut than for learning the visual corrections required by the vision-critical minority. Counterfactual obstacle stitching further reveals weak visual responsiveness in three existing VLAs (Table 4), demonstrating the corresponding failure to adapt trajectories to changed visual conditions. Together, the measured imbalance and observed blind planning support our central diagnosis: trajectory supervision is dominated in sample count by ego-predictable cases, leaving vision-critical learning concentrated in a small subset. This motivates our two-part intervention: explicit future-state supervision and a decoder conditioned on visual states. The ablations below test their effects on trained models.

D.1 Direct Regression Loss Decomposition

We derive the bias–variance decomposition of direct trajectory regression stated in Eq. (6) of Sec. 3. Under the majority-regime assumption, this decomposition shows that ignoring visual features is an optimal solution to the regime-specific loss, exposing the shortcut admitted by direct regression.

Notation. Let V_t denote the output of a frozen visual encoder, E the ego status, C the text command, and $\mathbb{I} \in \{0, 1\}$ the vision-critical indicator. We continue to use the target trajectory decomposition $T_t = T_0(E, C) + \mathbb{I} \cdot \Delta(V, E, C) + \xi$ (Eq. (3)) and the predictor decomposition

$$\bar{f}_\theta(E, C) \triangleq \mathbb{E}[f_\theta(V, E, C) \mid E, C, \mathbb{I}=0], \quad \tilde{f}_\theta(V, E, C) \triangleq f_\theta(V, E, C) - \bar{f}_\theta(E, C). \quad (11)$$

Since $\bar{f}_\theta(E, C)$ is by definition the conditional expectation of f_θ given $(E, C, \mathbb{I}=0)$, it is constant given (E, C) , and by linearity of conditional expectation $\mathbb{E}[\tilde{f}_\theta \mid E, C, \mathbb{I}=0] = \mathbb{E}[f_\theta \mid E, C, \mathbb{I}=0] - \bar{f}_\theta(E, C) = 0$, i.e., $\tilde{f}_\theta$ is the conditional residual relative to that mean. We additionally assume that the noise ξ is zero-mean and (conditionally) independent of (E, C, V) and $\tilde{f}_\theta$.

Step-by-step derivation of the bias–variance decomposition. Under $\mathbb{I} = 0$, the target is $T_0(E, C) + \xi$, so we expand the conditional MSE as follows:

$$\mathcal{L}_0(\theta) = \mathbb{E}[\|T_0(E, C) + \xi - f_\theta(V, E, C)\|_2^2 \mid E, C, \mathbb{I}=0] \quad (12)$$

$$= \mathbb{E}[\|(T_0(E, C) - \bar{f}_\theta(E, C)) - \tilde{f}_\theta(V, E, C) + \xi\|_2^2 \mid E, C, \mathbb{I}=0] \quad (13)$$

$$= \|T_0(E, C) - \bar{f}_\theta(E, C)\|_2^2 + \mathbb{E}[\|\tilde{f}_\theta\|_2^2 \mid E, C, \mathbb{I}=0] + \mathbb{E}[\|\xi\|_2^2]. \quad (14)$$

(13) is obtained by simply substituting $f_\theta = \bar{f}_\theta + \tilde{f}_\theta$ (Eq. (11)). In the transition to (14), the three cross terms that arise are all expanded as inner products and then cancelled as follows:

- $-2 \mathbb{E}[\langle T_0(E, C) - \bar{f}_\theta(E, C), \tilde{f}_\theta \rangle \mid E, C, \mathbb{I}=0] = 0$: because $(T_0 - \bar{f}_\theta)$ is constant given (E, C) and $\mathbb{E}[\tilde{f}_\theta \mid E, C, \mathbb{I}=0] = 0$.
- $2 \mathbb{E}[\langle T_0(E, C) - \bar{f}_\theta(E, C), \xi \rangle \mid E, C, \mathbb{I}=0] = 0$: for the same reason, $(T_0 - \bar{f}_\theta)$ is constant and $\mathbb{E}[\xi \mid E, C, \mathbb{I}=0] = 0$.
- $-2 \mathbb{E}[\langle \tilde{f}_\theta, \xi \rangle \mid E, C, \mathbb{I}=0] = 0$: by the assumption that ξ is zero-mean and (conditionally) independent of $\tilde{f}_\theta$.

This treatment of cross terms is the same as that used in the next-state predictor analysis (Eq. (18)–Eq. (19)) and the IK Network analysis (Eq. (26)) of this section.

D.2 Next Visual State Prediction

In this section, we mathematically analyze how the next visual state prediction objective introduced in Sec. 3 penalizes visual collapse. Through a bias–variance decomposition of the next visual state prediction loss $\mathcal{L}_{\text{state}}$, we show that ignoring visual features leaves scene-dependent prediction error that an ego-only predictor cannot reduce. When the model can predict this residual variation from current visual features, visual collapse is *strictly suboptimal*: reducing the loss further requires using visual information beyond (E, C) .

Notation. For brevity, let $V_t \triangleq \text{enc}(V_t)$ denote the output of a frozen visual encoder, E the ego status, C the text command, and $V_{t+\Delta}$ the visual state target token Δ seconds in the future. We denote the LLM’s next-state predictor as $w_\phi(V_t, E, C)$ and retain the vision-critical indicator $\mathbb{I} \in \{0, 1\}$ defined above. This indicator is not an input mask; the visual stream is available in both regimes.

Bias–variance decomposition. We define the conditional means and residuals of the predictor and target as follows:

$$\bar{w}_\phi(E, C) \triangleq \mathbb{E}[w_\phi(V_t, E, C) \mid E, C, \mathbb{I}=0], \quad \tilde{w}_\phi \triangleq w_\phi(V_t, E, C) - \bar{w}_\phi(E, C), \quad (15)$$

$$\bar{V}(E, C) \triangleq \mathbb{E}[V_{t+\Delta} \mid E, C, \mathbb{I}=0], \quad \tilde{V} \triangleq V_{t+\Delta} - \bar{V}(E, C). \quad (16)$$

By definition, $\mathbb{E}[\tilde{w}_\phi \mid E, C, \mathbb{I}=0] = 0$ and $\mathbb{E}[\tilde{V} \mid E, C, \mathbb{I}=0] = 0$.

Using these, the next-state loss $\mathcal{L}_{\text{state},0}$ (the loss defined under $\mathbb{I} = 0$) can be expanded as:

$$\mathcal{L}_{\text{state},0} = \mathbb{E}[\|V_{t+\Delta} - w_\phi(V_t, E, C)\|_2^2 \mid E, C, \mathbb{I}=0] \quad (17)$$

$$= \mathbb{E}[\|(\bar{V} - \bar{w}_\phi) + (\tilde{V} - \tilde{w}_\phi)\|_2^2 \mid E, C, \mathbb{I}=0] \quad (18)$$

$$= \|\bar{V}(E, C) - \bar{w}_\phi(E, C)\|_2^2 + \mathbb{E}[\|\tilde{V} - \tilde{w}_\phi\|_2^2 \mid E, C, \mathbb{I}=0]. \quad (19)$$

In (18) we have simply substituted $V_{t+\Delta} = \bar{V} + \tilde{V}$ and $w_\phi = \bar{w}_\phi + \tilde{w}_\phi$. In the transition to (19), the cross term cancels because $(\bar{V} - \bar{w}_\phi)$ is constant given (E, C) and, by construction, both $\tilde{V}$ and $\tilde{w}_\phi$ are residuals relative to their conditional means: $\mathbb{E}[\tilde{V} \mid E, C, \mathbb{I}=0] = 0$ and $\mathbb{E}[\tilde{w}_\phi \mid E, C, \mathbb{I}=0] = 0$. Hence by linearity of conditional expectation, $\mathbb{E}[\tilde{V} - \tilde{w}_\phi \mid E, C, \mathbb{I}=0] = 0$. That is, the loss neatly splits into a *bias term* $\|\bar{V} - \bar{w}_\phi\|_2^2$ and a *conditional variance term* $\mathbb{E}[\|\tilde{V} - \tilde{w}_\phi\|_2^2]$.

Lower bound under visual collapse. We now define “visual collapse” as the state in which the LLM does not effectively make use of V_t , i.e., the state in which the output of w_ϕ is invariant with respect to V_t :

$$w_\phi(V_t, E, C) \approx g_\phi(E, C) \quad \forall V_t \iff \tilde{w}_\phi \approx 0. \quad (20)$$

We also reflect the fact that, even under the same (E, C) , the future visual state varies due to the inherent diversity of scenes, by defining the data-intrinsic visual diversity as

$$\sigma_V^2 \triangleq \mathbb{E}_{E,C}[\text{Var}(V_{t+\Delta} \mid E, C)] > 0. \quad (21)$$

Under these two definitions, when collapse occurs ($\tilde{w}_\phi = 0$) the bias–variance decomposition simplifies as follows:

$$\mathcal{L}_{\text{state},0} = \|\bar{V}(E, C) - \bar{w}_\phi(E, C)\|_2^2 + \mathbb{E}[\|\tilde{V} - \tilde{w}_\phi\|_2^2 \mid E, C, \mathbb{I}=0] \quad (22)$$

$$\geq \mathbb{E}[\|\tilde{V} - \tilde{w}_\phi\|_2^2 \mid E, C, \mathbb{I}=0] \quad (23)$$

$$= \mathbb{E}[\|\tilde{V}\|_2^2 \mid E, C, \mathbb{I}=0] \quad (24)$$

$$= \text{Var}(V_{t+\Delta} \mid E, C, \mathbb{I}=0). \quad (25)$$

(23) simply drops the bias term, which is non-negative; (24) is the result of substituting $\tilde{w}_\phi = 0$ from the collapse definition (20); finally, (25) is a direct consequence of $\bar{V}(E, C)$ being defined as a conditional mean, expressing the conditional variance.

The last equality bounds the conditional loss of a collapsed predictor; averaging it over (E, C) within the same regime yields the corresponding regime-specific variance bound. When the predictor can recover a nonzero component of this residual variation from V_t , using vision reduces the attainable loss below the best ego-only prediction, making visual collapse strictly suboptimal. This is the central difference from direct trajectory regression: NSP rewards predictive visual information even within the ego-predictable trajectory regime. A loss below the bound, evaluated with the same conditioning and feature representation, rules out visual collapse.

D.3 Inverse Kinematics Formulation

The IK Network connects future-state prediction to trajectory generation by decoding actions from the current and predicted visual states. Existing VLAs admit a direct ego-status shortcut in trajectory regression (Sec. 3); our decoder removes this bypass and receives task context through the supervised state interface. Together with NSP supervision and trajectory-gradient isolation, this interface makes state prediction the representation on which action decoding operates.

Decomposition of the trajectory loss. Our IK Network h_ψ decodes the trajectory T taking only the current visual state V_t and the next visual state $\hat{V}_{t+\Delta}$ predicted by the LLM as input. Letting $T_0(E, C)$ denote the mean of the “ideal” trajectory determined from (E, C) , and ξ the zero-mean noise inherent in the trajectory, the trajectory loss under $\mathbb{I} = 0$ is decomposed as follows:

$$\begin{aligned} \mathcal{L}_{\text{traj},0}(\psi) &= \mathbb{E} \left[\|T_0(E, C) + \xi - h_\psi(V_t, \hat{V}_{t+\Delta})\|_2^2 \mid E, C, \mathbb{I}=0 \right] \\ &= \underbrace{\|T_0(E, C) - \bar{h}_\psi(E, C)\|_2^2}_{\text{bias}} + \underbrace{\mathbb{E} [\|\tilde{h}_\psi\|_2^2 \mid E, C, \mathbb{I}=0]}_{\text{variance}} + \underbrace{\mathbb{E} [\|\xi\|_2^2]}_{\text{irreducible noise}}. \end{aligned} \quad (26)$$

Here, $\bar{h}_\psi(E, C)$ and $\tilde{h}_\psi$ are the conditional mean and residual defined in the same manner as for w_ϕ .

While the *form* of the equation is identical to that of the decomposition obtained for the existing structure, the *coupling* between the two terms is decisively different.

Decisive difference between the existing and the proposed structures. In existing VLAs, since the trajectory predictor f_θ takes (V_t, E, C) all directly as input, the conditional mean $\bar{f}_\theta(E, C)$ can be computed using only the (E, C) token pathway. Hence, even if the model chooses $\tilde{f}_\theta = 0$ (ignoring visual features), the learning of $\bar{f}_\theta$ is unaffected, and the two terms can be minimized *independently*, making it a valid optimal solution to ignore visual features.

In contrast, our IK Network h_ψ does not take (E, C) directly as input and only takes $(V_t, \hat{V}_{t+\Delta})$. The decoder therefore receives task information through the current and predicted visual-state tokens. The predicted state $\hat{V}_{t+\Delta} = w_\phi(V_t, E, C)$ integrates scene information with the ego and command context needed for planning. The stop-gradient interface in Sec. E prevents the trajectory loss from adapting these tokens, while NSP supervises their agreement with future visual features.

Fully ignoring both conditioning inputs forces the decoder to output a single constant c :

$$h_\psi(V_t, \hat{V}_{t+\Delta}) = c \quad \text{for all conditioning pairs} \implies \bar{h}_\psi(E, C) = c. \quad (27)$$

Then the best attainable average bias is

$$\min_c \mathbb{E} [\|T_0(E, C) - c\|_2^2 \mid \mathbb{I} = 0] = \text{Var}(T_0(E, C) \mid \mathbb{I} = 0) > 0, \quad (28)$$

provided the target conditional mean varies across (E, C) in this regime. These equations quantify the cost of discarding the entire conditioning pair: fitting the varying target trajectory requires using information carried by the state tokens. Here, full conditioning collapse means a globally constant output; $\tilde{h}_\psi = 0$ instead describes zero residual at fixed (E, C) . Our design combines this state-conditioned decoding with NSP supervision and trajectory-gradient isolation. Stitching measures the resulting visual responsiveness, and the controlled gradient-flow ablation demonstrates the safety benefit of preserving the supervised state interface (Tables 4 and 8).

D.4 Empirical Estimates of ε and σ_V^2 on nuScenes

The arguments in Secs. D.1 and D.2 hinge on two scalar quantities that we have so far treated only symbolically. The first is the fraction $\varepsilon \triangleq \mathbb{P}(\mathbb{I}_t = 1)$ of vision-critical samples in the trajectory decomposition $T_t = T_0(E, C) + \mathbb{I}_t \cdot \Delta(V, E, C) + \xi$ (Eq. (3)), which sets the weight of the only term in the mixture loss (Eq. (4)) that pushes the trajectory predictor to depend on V . The second is the data-intrinsic visual diversity $\sigma_V^2 = \mathbb{E}_{E,C} [\text{Var}(V_{t+\Delta} \mid E, C)] > 0$ from Eq. (21), the lower bound below which the next-state loss cannot drop without using V_t (Eq. (25)). This section reports empirical proxies for these quantities on the nuScenes validation split using the DINOv3 ViT-S/16 features adopted in our experiments (front-left, front, front-right; 540 tokens of dimension 384 per frame; $\Delta = 3$ s, equivalently 6 keyframes ahead).

Estimating $\varepsilon = \mathbb{P}(\mathbb{I}_t = 1)$. We follow the operational definition used in our ego-status stratified evaluation (Sec. O): for each validation sample we extrapolate the future trajectory by integrating constant linear velocity and constant yaw rate from `gt_ego_lcf_feat`, and define the *ego-status gap* as the 3-second ADE between this kinematic prediction and the GT trajectory. We operationalize the vision-critical subset using an ego-status gap of at least τ , yielding a measurable proxy for the latent regime indicator in Eq. (10). The marginal distribution of the gap on the nuScenes val split ($n = 6,019$) has mean 2.230 m, median 1.286 m, 70th percentile 2.139 m, and 90th percentile 4.888 m. Table 6 reports $\hat{\varepsilon}(\tau) = \frac{1}{n} \sum_i \mathbf{1}[\text{gap}_i \geq \tau]$ for several thresholds.

Table 6: Empirical $\hat{\varepsilon}$ on the nuScenes val split ($n = 6,019$) at several gap thresholds. The threshold $\tau = 2.14$ m corresponds to the top-30% boundary used in our ego-status stratification (Sec. O).

max width=	τ (m)	0.50	1.00	1.50	2.00	2.14	3.00	5.00
	$\hat{\varepsilon}$	0.763	0.584	0.442	0.328	0.300	0.187	0.098

At $\tau = 2.14$ m the proxy fraction is 0.300, and at $\tau = 5$ m it is 0.098. Across these thresholds, the proxy-defined vision-critical subset accounts for only 10–30% of samples, leaving a 70–90% majority with smaller kinematic errors. This sample imbalance concentrates the trajectory-supervision incentive for visual corrections in a minority of training examples. NSP addresses this imbalance by supervising scene-dependent future visual content across both trajectory regimes.

Estimating $\sigma_V^2 = \mathbb{E}_{E,C}[\text{Var}(V_{t+\Delta} \mid E, C)]$. For each val token t we look up the token $\Delta = 6$ keyframes ahead via repeated `next` traversal of the cached info dict, and accumulate the per-element mean and second moment of the future-frame DINOv3 feature $V_{t+\Delta} \in \mathbb{R}^{540 \times 384}$ using Welford’s algorithm. Of the 6,019 val tokens, 5,119 have both the current and the +3 s feature cached and are usable for this estimate. Since (E, C) are continuous and high-dimensional, we use the discrete 3-way `gt_ego_fut_cmd` (right / forward / left) as a tractable proxy for C , and report two numbers: the marginal $\text{Var}(V_{t+\Delta})$, and the conditional proxy $\mathbb{E}_C[\text{Var}(V_{t+\Delta} \mid C)]$.

Table 7: Variance of the future DINOv3 feature $V_{t+\Delta}$ on the nuScenes val split ($n = 5,119$). Reported as $\|\cdot\|_2^2$ form: the sum of entry-wise variances over all $540 \times 384 = 207,360$ feature dimensions. The conditioning differs from that in Eq. (25), so these statistics are descriptive proxies.

max width=	quantity	$\ \cdot\ _2^2$ form
	Marginal $\text{Var}(V_{t+\Delta})$	1.162×10^4
	Conditional $\mathbb{E}_C[\text{Var}(V_{t+\Delta} \mid C)]$ (σ_V^2 proxy)	1.158×10^4
	Between-command variance	34

The ratio of command-conditioned to marginal variance is 0.997: the discrete route command explains only about 0.3% of the measured visual variability. The command-conditioned proxy is approximately 1.16×10^4 , or 5.6×10^{-2} per scalar dimension. This statistic measures visual diversity remaining within each command; the analytical bound in Eq. (25) additionally conditions on ego status and the regime.

The NSP residual in Table 18 is at most approximately 2.2×10^{-2} per element across the reported quartiles. This residual measures prediction fidelity in the model’s 896-dimensional projected representation, while the variance proxy measures diversity in raw 384-dimensional DINOv3 features. Testing the theoretical no-vision bound requires matched feature spaces and conditioning; the direct evidence for visual use comes from the controlled interventions below.

Joint reading. The ego-status gap distribution establishes a pronounced sample imbalance under the proxy, while the visual-feature statistics reveal substantial scene variation beyond the route command. These measurements explain the motivation for supervising future visual states: trajectory targets are often readily extrapolated from ego status, whereas visual targets retain scene-dependent content. Counterfactual stitching and controlled ablations then establish the contribution in model behavior: stronger hazard-specific responses and improved planning with the proposed state interface.

E Gradient Isolation at the IK Conditioning Interface

Stop-gradient conditioning. Restricting the IK Network’s forward inputs to visual-state tokens alone does not prevent trajectory-loss gradients from modifying the predicted state during joint optimization. Our implementation therefore applies stop-gradient to both conditioning inputs, the current visual state V_t and the predicted next visual state $\hat{V}_{t+\Delta t}$. Writing $\text{sg}(\cdot)$ for an operation that preserves its input in the forward pass and has zero derivative, the diffusion decoder receives

$$\epsilon_\psi \left(x_s, s, \text{sg}(V_t), \text{sg}(\hat{V}_{t+\Delta t}) \right), \quad \nabla_\phi \mathcal{L}_{ik} = 0, \quad (29)$$

where s denotes the diffusion timestep and ϕ denotes the LLM parameters. Thus, the IK trajectory loss updates the diffusion decoder but is never propagated to the LLM through its conditioning inputs. The predicted next visual state is directly supervised by the NSP MSE objective against ground-truth future DINOv3 features in the projected visual-token space, rather than by the trajectory loss. Auxiliary QA supervision can still update the shared LLM parameters, as described in Sec. B. This separation closes the trajectory-gradient channel through which the diffusion head could otherwise reshape the predicted state to encode a planning shortcut. It provides the same isolation from trajectory-loss updates as training the state predictor first and then training the action decoder with the predictor frozen; it does not require the two modules to be optimized in separate chronological stages.

Controlled gradient-flow ablation. To measure the effect of this design, we compare two models on the NAVSIM navtest split with identical data, training schedules, and architectures, changing only whether IK trajectory gradients are blocked at the conditioning interface or allowed to flow back to the LLM. Table 8 reports the results.

Table 8: Effect of IK trajectory-gradient flow to the LLM on NAVSIM navtest. The two variants differ only in whether gradients are blocked at the IK conditioning inputs. Higher is better for all metrics.

	max width=					
IK trajectory gradient to LLM	PDMS v1 $\uparrow$	EPDMS v2 $\uparrow$	NC $\uparrow$	DAC $\uparrow$	TTC $\uparrow$	EP $\uparrow$
Detached (ours)	90.29	89.25	99.18	96.22	99.00	87.60
Backflow allowed	86.24	84.91	97.11	93.89	96.65	88.11

Allowing gradient backflow reduces PDMS by 4.05 points and EPDMS by 4.34 points. The reported safety metrics all deteriorate: NC decreases by 2.07 points, DAC by 2.33 points, and TTC by 2.35 points, while ego progress remains similar, increasing slightly by 0.51 points. With data, architecture, and training schedule fixed, blocking trajectory gradients improves all three safety metrics and both aggregate planning scores while preserving comparable progress. This ablation establishes gradient isolation as an effective component of the state interface.

F Note on the OpenDriveVLA* Score in NAVSIM Benchmarks

OpenDriveVLA* records PDMS/EPDMS of 73.2/70.2 in Tables 1 and 2. It uses the same 0.5B Qwen2.5 backbone and DINOv3 ViT-S/16 encoder as our model, with matched data and training schedules, and preserves direct trajectory text output. This matched comparison isolates the combined NSP and IK design from differences in backbone scale, visual encoder, data, and training schedule: our model improves by 19.0 PDMS and 20.4 EPDMS. The broader benchmark tables additionally compare against VLAs with their reported backbone, encoder, and decoding configurations. OpenDriveVLA* is the matched NAVSIM implementation; the original nuScenes model uses a different BEV encoder. The benchmark tables identify these distinct configurations.

G Analytical Mechanism and Empirical Contributions

Analytical diagnosis and empirical validation. The decomposition in Sec. D identifies an ego-only optimum of the majority-regime loss under the stated conditional-independence assumption. This exposes the training incentive that our architecture addresses: a predictor can fit ego-predictable trajectories without learning the visual corrections needed in vision-critical scenes. NSP changes

this incentive by rewarding future visual content, and the IK interface connects that supervised representation to trajectory generation. The derivation and empirical regime calibration are given in Sec. D.

Behavioral evidence independent of the decomposition. The no-vision ego/command decoder in Table 9 reaches L2 0.60 m versus 0.53 m for ours under the ST-P3 protocol, yet responds to none of the obstacle insertions across 6,019 samples. This directly demonstrates why planning accuracy alone can conceal blind planning. In the shared-intervention evaluation (Table 16), our hazard–placebo AUC is 0.628, compared with 0.542 for original OpenDriveVLA. The matched conditioning ablation further increases the hazard–placebo gap from 5.8 to 37.6 percentage points when NSP is introduced. These measurements establish the behavioral contribution independently of the analytical assumptions.

Matched comparisons and benchmark performance. The NAVSIM comparison controls the visual encoder, 0.5B backbone, training data, and optimization schedule: introducing NSP and the IK Network improves over OpenDriveVLA* by 19.0 PDMS and 20.4 EPDMS. Conditioning and gradient-flow ablations identify the contributions within this design (Secs. E and I). Across Tables 1 to 3, our 0.5B model achieves state-of-the-art planning scores using cameras alone, without the LiDAR geometry available to C+L baselines or the explicit BEV encoder used by original OpenDriveVLA on nuScenes. Cross-family comparisons report each system’s benchmark performance under its own sensor, encoder, and training configuration; parameter counts denote the reported model or backbone scale, rather than matched total compute.

Retaining task information through the state interface. Removing direct ego-state access makes the predicted state responsible for delivering task context to the IK decoder. The probes recover ego speed with $R^2 = 0.99$ and command with 98.5% accuracy, and command interventions verify that this information controls trajectory direction (Sec. J). The interface therefore retains useful task information while removing the direct ego/command bypass. The input-removal and visual-intervention diagnostics show that the planner uses this task context together with visual evidence (Sec. J).

H Ablation Study

To verify the strengths of our methodology, we removed each of the three core components of our model one by one and analyzed their contributions (Table 5).

Effect of Next State Prediction. First, we removed *next state prediction*, which trains the LLM to predict the future ($\Delta t = 3$ s) visual state, and replaced it with *current state reconstruction* as the training objective. As a result, ADE degraded by +0.01, from 0.06 $\rightarrow$ 0.07. NSP’s contribution is clearer in the controlled hazard–placebo comparison: with the diffusion decoder fixed, the response-selectivity gap increases from 5.8 to 37.6 percentage points (Sec. I).

However, in additional analyses on scenes where the two models differ, we found that in dynamic driving situations that require more precise use of visual features, such as curve regions, the model trained with next state prediction consistently performs better (Table 17). That is, even if current state reconstruction performs similarly on average, it shows shortcomings in challenging scenes; a detailed analysis is provided in the supp. material (Table 17).

Effect of the IK Network. Second, we removed the *IK Network* that takes the current/next visual state as input and decodes the trajectory, and replaced it with a small MLP head applied directly to the LLM’s output hidden state. As a result, ADE degraded by +0.25, from 0.06 $\rightarrow$ 0.31, i.e., a nearly $5\times$ larger gap. This suggests that when the trajectory is regressed directly from the LLM hidden state, the model is again exposed to the shortcut learning problem. In contrast, our IK Network separates trajectory generation into a separate module and restricts its inputs *only to the visual state*, thereby forcing the LLM to learn scene understanding before memorizing trajectories.

Architecture of the IK Network. Third, to verify the structural design choices of the IK Network itself, we replaced the diffusion-based IK Network with a *non-generative* variant. Specifically, we changed it to a structure that takes the current/next visual state as keys and values, performs cross-attention pooling with a learnable query, and then *directly* regresses the trajectory through an MLP regression head. As a result, ADE degraded by +0.10, from 0.06 $\rightarrow$ 0.16. This suggests two things.

First, since this variant still preserves the core design of next state prediction and a visual-only IK module, an ADE of 0.16 is still overwhelmingly better than *w/o IK Network* (0.31). The advantage persists with deterministic decoding, showing that the benefit of the state-conditioned IK design extends beyond the diffusion decoder. Second, even within the same structure, implementing the IK module as a *generative diffusion* rather than a deterministic regression elevates trajectory precision one step further, which is a natural result given that the trajectory distribution is inherently multi-modal and stochastic. The ablation table is presented in the main paper (Table 5).

I Disentangling Next-State Prediction from the Visual Diffusion Decoder

Motivation and controlled protocol. The main ablation (Table 5) shows a small ADE increase when NSP is replaced by current-state reconstruction ($0.06 \rightarrow 0.07$ m), compared with the larger degradation when the IK Network is removed ($0.06 \rightarrow 0.31$ m). To isolate NSP’s contribution to visual grounding, we hold the diffusion decoder fixed and measure both trajectory accuracy and hazard-specific response. We therefore conduct a separate, unified four-epoch ablation on nuScenes, training every variant from scratch with its ablation active from the beginning and evaluating on all 6,019 validation samples. The main-table training comprises four nuScenes epochs plus one additional epoch on the language dataset; this ablation omits the additional language epoch to remove its contribution from the comparison. All variants use the same diffusion IK head and training schedule; only the conditioning configuration and corresponding state-supervision target change. Because this series excludes the additional language training used for the main-table models, its absolute results are reported separately; comparisons between conditioning variants use only this unified series.

Conditioning variants. Row (0) zeros the cross-attention keys and values and provides neither visual conditioning nor ego status or command. Row (0+) retains this absence of visual conditioning but injects ego status and command directly into the decoder. Row (i) uses current-state reconstruction instead of NSP, providing current-state information only; the decoder architecture and conditioning-token count are matched to row (ii). Row (ii) is our predicted-future configuration, conditioned on $(V_t, \hat{V}_{t+\Delta})$. Row (iii) replaces the predicted future with the ground-truth future visual feature, using $(V_t, V_{t+\Delta}^{\text{GT}})$ during both training and evaluation. Row (iii) requires an unavailable future observation at deployment and is included as a diagnostic.

Hazard and placebo measurements. For each scene, we compare an unmodified input with an input containing a synthetic vehicle ahead, keeping the inference seed fixed between the paired runs, following the obstacle-stitching protocol in Table 4. The hazard deceleration rate is the fraction of scenes in which the predicted trajectory shortens after insertion. We repeat the comparison with the same sprite inserted into the sky as a placebo. The hazard–placebo gap measures response selectivity in this intervention: a large hazard response accompanied by a similarly large sky response is weak evidence of hazard-specific grounding. Mean Δ displacement is reported for the hazard intervention, with negative values indicating shorter trajectories.

Table 9: Controlled conditioning ablation on nuScenes validation ($n = 6,019$), with each variant trained from scratch for four epochs. L2 averages are reported under the ST-P3 and UniAD evaluation protocols. Hazard and sky rates are percentages; their difference is in percentage points. Lower L2 and a larger hazard–sky gap are desirable; a larger displacement reduction is not necessarily better.

IK conditioning	max width=				
	L2 avg. (m) ↓ ST-P3 / UniAD	Hazard (%)	Sky (%)	Gap (pp) ↑	Mean Δ displ. (m)
(0) No conditioning (K/V zeroed)	5.58 / 8.62	0.0	0.0	0.0	0.000
(0+) No vision; direct ego/command	0.60 / 1.11	0.0	0.0	0.0	0.000
(i) Current-state information only	0.55 / 1.01	71.3	65.5	5.8	−0.21
(ii) $V_t + \hat{V}_{t+\Delta}$ (ours)	0.53 / 0.98	82.8	45.2	37.6	−0.62
(iii) $V_t + V_{t+\Delta}^{\text{GT}}$	2.58 / 4.05	95.4	46.1	49.3	−8.71

Visual conditioning and accuracy without grounding. Without any conditioning, row (0) has L2 errors of 5.58/8.62 m. Providing only ego status and command in row (0+) reduces these errors

to 0.60/1.11 m despite zero response to either visual intervention. Thus, relatively low trajectory error is attainable without visual responsiveness, making L2 alone insufficient to establish grounding. Adding current-state conditioning in row (i) improves accuracy and introduces a visual response, but its small hazard–placebo gap indicates limited selectivity.

NSP adds response selectivity beyond the same diffusion decoder. The controlled comparison between rows (i) and (ii) holds the diffusion architecture and token count fixed. NSP improves L2 from 0.55/1.01 to 0.53/0.98 m, raises the hazard deceleration rate from 71.3% to 82.8%, and reduces the sky-placebo rate from 65.5% to 45.2%. The hazard–placebo gap increases from 5.8 to 37.6 percentage points, approximately $6.5\times$, while the magnitude of mean hazard-induced shortening increases from 0.21 to 0.62 m. With the diffusion decoder and token count fixed, this $6.5\times$ increase establishes NSP’s contribution to hazard-specific response selectivity. The direction of the accuracy change agrees with the main ablation’s ADE increase from 0.06 to 0.07 m when NSP is removed (Table 5). Among the deployable configurations, row (ii) combines the lowest L2 error with the largest hazard–placebo gap. The improvement combines a higher hazard response with a lower placebo response; the remaining 45.2% placebo rate quantifies residual sensitivity to irrelevant visual changes.

Predicted future states preserve planning-relevant context. Row (iii) achieves a high hazard deceleration rate (95.4%), but its L2 errors rise to 2.58/4.05 m and the mean displacement change reaches -8.71 m. This combination is consistent with excessive trajectory shortening rather than improved planning, despite a large hazard–placebo gap. Thus, raw future visual features do not serve as an upper bound for this decoder, and intervention response rates must be interpreted alongside trajectory accuracy and response magnitude.

To examine the difference between predicted and ground-truth future representations, we report linear probes for ego speed and command in Table 10. The predicted representation supports ego-speed prediction with $R^2 = 0.99$ and command classification with 98.5% accuracy, compared with 0.33 and 79.5% for the ground-truth future feature. The latter command accuracy is below the 88.2% majority-class baseline. These probes show that ego-kinematic and goal information is substantially more linearly accessible in the predicted state than in the ground-truth visual feature. The NSP predictor receives (E, C) , while the IK decoder has no direct access to them; replacing its predicted state can therefore change the accessibility of task information even when the replacement is an actual future observation. The probes measure the linear accessibility of task information, providing a representation-level explanation consistent with the trajectory degradation in row (iii). Our aim is to suppress planning that bypasses visual evidence, while retaining the ego and command information needed for the requested maneuver.

Table 10: Linear probing of predicted and ground-truth future features. The input-independent reference is a mean predictor for ego speed and a majority-class predictor for command.

Probe target	Predicted $\hat{V}_{t+\Delta}$	GT $V_{t+\Delta}$	Input-independent baseline
Ego speed (R^2)	0.99	0.33	0
Command accuracy	98.5%	79.5%	88.2%

J Diagnosing Ego-Status and Command Information in the Predicted Future State

Visual prediction with task context. The future-state predictor integrates visual observations with ego status, command, history, and previous actions to construct the representation consumed by the IK decoder. Command information selects the requested maneuver, and ego kinematics specify its starting motion; retaining this context is essential to task-conditioned planning. We evaluate how the representation combines this context with visual evidence through representation probes, selective input removal, command interventions, and a decoder without visual inputs. Together, these diagnostics assess both the content of the predicted state and its measured influence on trajectory generation.

J.1 Forward and Reverse Representation Probes

Shared protocol and forward probes. Probes are fit on 8,000 nuScenes training samples and evaluated on all 6,019 validation samples, with no sample-token overlap between the splits. The VLA is frozen and run only in the forward direction. As reported in Table 10, ego speed is linearly decoded from the predicted future state with $R^2 = 0.99$, and command with 98.5% accuracy. Thus, ego and command information are retained, as expected for a task-conditioned future representation.

Reverse probes: predicting future visual content. We next predict a common visual target from either (E, C) or pooled features of the predicted future state consumed by the IK head. The target is the global mean of the ground-truth next-frame DINOv3 tokens (384 dimensions), a summary of the future visual observation used for NSP supervision. The latent predictors use a 3,584-dimensional pooled representation or a 14,336-dimensional representation obtained with 16-cell pooling. The ego/command baselines use seven input dimensions, with either a second-order polynomial ridge model (35 expanded dimensions) or non-parametric k -nearest-neighbor regression ($k = 25$). For ridge probes, regularization is selected on an internal development split of the training data; validation is reserved for evaluation. We report the train-mean-referenced score

$$R_{\text{train-mean}}^2 = 1 - \frac{\sum_{i \in \text{val}} \|y_i - \hat{y}_i\|_2^2}{\sum_{i \in \text{val}} \|y_i - \bar{y}_{\text{train}}\|_2^2}, \quad (30)$$

so predicting the training mean scores zero and negative values indicate worse performance than that baseline.

Table 11: Reverse probes predicting the global mean of ground-truth future DINOv3 features. All results use the held-out nuScenes validation set ($n = 6,019$). Pooling is applied for probing to the predicted-state tokens supplied to the IK head.

	Predictor	$R_{\text{train-mean}}^2$
max width=	Ego status + command: polynomial ridge (35-d)	0.017
	Ego status + command: k NN ($k = 25$)	-0.061
	Predicted future state: pooled (3,584-d)	0.423
	Predicted future state: 16-cell pooling (14,336-d)	0.599
	Predicted future state: permuted pairing control	-0.037

Under this common target and evaluation protocol, the pooled predicted state explains 42.3% of squared-error variation relative to the train-mean baseline, compared with 1.7% for the polynomial ego/command baseline. Destroying the input-target pairing by permutation yields $R^2 = -0.037$, supporting the importance of scene-specific correspondence rather than feature dimensionality alone. When the target instead preserves partial spatial structure through 16 cells, the reported ego/command and predicted-state scores are 0.010 and 0.244, respectively. These targets summarize the frame rather than reconstruct every token. Together with the forward probes, these results indicate that the latent retains both task context and predictive visual content beyond what the tested ego/command models recover. The comparison isolates the predictive content recovered from ego status and command; history and previous actions are outside the inputs of these probe baselines.

J.2 Selective Removal and Command Interventions

Removing predictor inputs at inference. We remove either command C or ego status E (both its textual representation and `can_bus` fields) during future-token prediction, retain the deployed IK head, and compare paired runs with the same seed on all 6,019 validation samples. The seed-only control changes the decoding seed without removing inputs.

Removing command has a modest overall effect but a larger effect on turns (7.21%) than on straight-driving scenes (0.37%). The dataset contains 5,306 keep-forward commands out of 6,019 samples, which helps explain why an aggregate score can obscure maneuver-dependent effects. Removing ego status has a larger effect in both subsets (54.0% on turns and 43.3% on straight-driving scenes), consistent with its role in specifying motion. The seed-only changes are negligible by comparison. These paired interventions measure the deployed model’s reliance on each input. They are inference-time removals, so their effects include the input-distribution shift induced by removal.

Table 12: Effect of removing inputs to the future-state predictor at inference. Positive changes indicate higher trajectory error. Turning and straight columns give relative ADE changes in their respective subsets.

Removed input	max width=			
	Overall Δ ADE (m)	Overall (%)	Turning (%)	Straight (%)
Command C	+0.008	+1.60	+7.21	+0.37
Ego status E (text + can_bus)	+0.227	+45.2	+54.0	+43.3
Seed change only (control)	−0.0001	−0.01	−0.06	−0.00

Changing the command while fixing the scene. We hold the image, ego status, and remaining context fixed, substitute left- and right-turn commands, and decode with the same seed. Direction agreement is the fraction of scenes in which the endpoint under the left command lies to the left of the endpoint under the right command. Table 13 reports this comparison and a same-command re-decoding control.

Table 13: Command interventions on the predicted future state. Endpoint shifts are measured at 3 s. A dash denotes an unreported quantity.

Subset	max width=		
	Direction agreement	Endpoint shift under flip	Same-command re-decoding
All scenes ($n = 6,019$)	99.73%	0.340 m	0.0011 m
Turning ($n = 1,498$)	100.00%	0.454 m	−
Moving ($n = 4,360$)	100.00%	−	−

Mean lateral endpoints under left, straight, and right commands are -0.170 , $+0.158$, and $+0.442$ m, respectively, while mean longitudinal distances remain similar at 14.48, 14.43, and 14.46 m. The 16 direction-agreement exceptions are stationary scenes with left/right separation of at most 12 mm. Because V_t is fixed and the IK decoder receives only $(V_t, \hat{V}_{t+\Delta})$, this intervention locates command influence in the predicted-state pathway. This isolates the command-conditioning route through the predicted state; the visual-intervention tests below measure the planner’s response to scene evidence.

J.3 Behavioral Controls for Visual Dependence

A decoder with no visual input. Row (0+) of Table 9 supplies ego status and command directly to the decoder while excluding vision. It achieves L2 0.60/1.11 m, close to our 0.53/0.98 m in the same four-epoch series, but has zero hazard and placebo response on all 6,019 samples. Our model responds to hazard insertions in 82.8% of scenes, with a 45.2% sky-placebo rate. This behavior is inconsistent with a planner whose output depends exclusively on unchanged non-visual context. This control tests visual dependence of the complete planner, which receives both V_t and $\hat{V}_{t+\Delta}$. The matched current-only versus predicted-future comparison in Sec. I then isolates the additional selectivity delivered by NSP.

Matched comparison with the original VLA. In a separate matched evaluation, we run our model, original OpenDriveVLA, and the non-visual decoder control on the same scenes, pixel interventions, and seeds (Table 14). We report these measurements separately from the four-epoch conditioning series. Our hazard–placebo gap is 14.5 percentage points versus 9.2 for original OpenDriveVLA, an increase of 5.3 points. Sec. L extends this matched comparison to four VLAs with same-pixel null controls and threshold-free hazard–placebo AUC. The position and size interventions in Table 21 provide additional evidence that the response depends on the spatial configuration of the visual change.

Joint evidence for the conditioning mechanism. The probes show that the predicted state carries ego/command information and predictive visual content; the interventions show that task context affects maneuver selection and that pixels affect trajectories. Together, these matched probes and behavioral controls demonstrate that the state interface retains useful task context and delivers visual information that affects planning. The controlled NSP ablation further identifies the gain in hazard-specific selectivity supplied by future-state prediction (Sec. I).

Table 14: Matched hazard and sky-placebo evaluation on nuScenes validation ($n = 6,019$), separate from the four-epoch conditioning ablation. Rates are percentages and gaps are percentage points.

Model	Hazard deceleration	Sky placebo	Gap
Ours	62.8%	48.3%	14.5
Original OpenDriveVLA	44.9%	35.7%	9.2
(0+) Decoder without vision	0.0%	0.0%	0.0

K Roles of Language and Ego Status in Future-State Prediction

Why language acts upstream of the IK decoder. The future-state predictor combines current visual tokens, ego status, and a textual route command, together with the supplied history and previous-action context. The intended roles are complementary: visual tokens describe the observed scene, the command selects a desired maneuver, and ego status specifies the current motion from which that maneuver begins. The predicted state then serves as a task-conditioned representation for the IK decoder. Thus, excluding a direct text input from the IK Network does not exclude language from planning; command information reaches the decoder through $\hat{V}_{t+\Delta}$. Here, the experimentally manipulated textual input is the route command, not an arbitrary scene description or the entire prompt. The diagnostics below assess route-command conditioning and ego-state use; Sec. R separately evaluates QA supervision.

Evidence that command information affects planning. The paired command interventions in Table 13 hold the image and ego status fixed and use the deployed IK decoder with the same seed. Left–right endpoint direction agrees with the command in 99.73% of all 6,019 validation scenes, and in all reported turning and moving scenes. The mean endpoint shift is 0.340 m, compared with 0.0011 m under same-command re-decoding. Mean lateral endpoints vary monotonically from left to straight to right, while the corresponding mean longitudinal distances remain similar (Sec. J). The 0.340 m shift, together with its consistent direction and the 0.0011 m re-decoding floor, demonstrates command control over the predicted-state pathway. The measurement concerns the 3-second endpoint response.

Selective input removal by the recorded route command. In addition to the maneuver-based breakdown in Table 12, we group command-removal effects by each scene’s recorded route command (Table 15). The relative ADE increase is 17.1% for left commands and 25.2% for right commands, compared with 0.15% for keep-forward commands. This grouping differs from the turning/straight scene subsets used in Table 12; their percentages should not be interchanged. The predominance of keep-forward commands (5,306 of 6,019 samples) helps explain why the overall command-removal effect is only 1.60% despite larger effects for commanded turns.

Table 15: Relative ADE increase when the route command is removed during future-state prediction, grouped by the scene’s recorded command. Paired inference uses the deployed IK head and the same seed on the full nuScenes validation set.

Recorded command	Left turn	Right turn	Keep forward
ADE change (%)	+17.1	+25.2	+0.15

Ego kinematics and the combined representation. Removing ego status from both the prompt text and can_bus inputs increases overall ADE by 45.2% (+0.227 m), with substantial effects in both turning and straight-driving subsets (Table 12). This broader dependence differs from the maneuver-selective effect of command removal. The forward probes in Table 10 recover ego speed from the predicted state with $R^2 = 0.99$ and command with 98.5% accuracy, versus 0.33 and 79.5% from the ground-truth future feature; the command majority-class baseline is 88.2%. Together, the interventions and probes show that the full model retains and uses both inputs in its conditioning representation. Each removal experiment measures an input’s contribution with the other available. Joint-removal and retraining experiments would further characterize their interaction (Sec. U).

The LLM as a shared visual and task-context predictor. Our LLM provides a shared sequence-processing backbone for visual tokens, structured ego information, route commands, and auxiliary language tasks. The probes and interventions demonstrate that this backbone integrates task context and predictive visual content into a representation used by the action decoder. The iteration-matched QA ablation establishes the planning benefit of language-task co-training: removing QA increases L2 from 0.06 to 0.38 m and doubles the collision rate (Sec. R). The four-epoch conditioning series separately isolates the contribution of the state interface without the additional language-dataset epoch used for the main-table model (Sec. I). A matched comparison with a non-language future-state generator remains future work (Sec. U).

L Architecture-Agnostic Evaluation of Visual Grounding

Why a shared behavioral evaluation is needed. The GradCAM comparisons use different differentiation targets: a diffusion denoising objective for our model, token cross-entropy for autoregressive baselines, and a proxy logit for Alpamayo-R1 (Sec. P.3). GradCAM localizes sensitivity within each architecture. Our primary cross-architecture evaluation uses shared pixel interventions and measures their effects directly on predicted trajectories. Hazard, placebo, and null controls distinguish obstacle-relevant responses from irrelevant visual sensitivity and decoding variability, giving a common behavioral test across architectures.

Common intervention protocol. We extend the obstacle-stitching evaluation with placebo and null controls and apply the same protocol to all four models on the full nuScenes validation set ($n = 6,019$). Each model receives identical synthesized image pixels for each intervention, while ego status, command, history, and other non-visual context are held fixed. The hazard arm inserts a vehicle ahead of the ego vehicle; the placebo arm places the same sprite in the sky. The null arm re-runs the identical, unmodified input to measure the model’s own decoding variability. We compare the resulting trajectories with their unmodified-input counterparts. The evaluation uses only input interventions and output trajectories, and therefore does not require access to a particular architecture, attention map, or latent representation.

Threshold-free hazard–placebo separation. Response rates count the fraction of scenes whose predicted trajectory shortens under each intervention. Their interpretation depends on the shortening threshold, and model rankings can change when response magnitudes differ. We therefore report hazard-versus-placebo AUC as the primary comparison, alongside hazard, placebo, and null response rates. Let s_i^{haz} and s_j^{sky} denote signed trajectory-shortening scores relative to the corresponding unmodified trajectories, with larger scores indicating stronger shortening. The empirical AUC is

$$\text{AUC} = \frac{1}{n^2} \sum_{i=1}^n \sum_{j=1}^n \left[\mathbf{1}(s_i^{\text{haz}} > s_j^{\text{sky}}) + \frac{1}{2} \mathbf{1}(s_i^{\text{haz}} = s_j^{\text{sky}}) \right]. \quad (31)$$

Thus, AUC estimates the probability that a randomly selected hazard response exceeds a randomly selected placebo response, assigning half credit to ties. An AUC of 0.5 indicates chance-level ranking, while 1.0 indicates perfect separation in the intended direction. This comparison avoids choosing a single response threshold, although it remains specific to the chosen intervention and response score.

Table 16: Architecture-agnostic grounding evaluation on all 6,019 nuScenes validation scenes, using identical synthesized pixels across models. AUC compares hazard and sky-placebo shortening scores; response-rate columns use the same intervention protocol.

	Model	Hazard response $\uparrow$	Sky placebo $\downarrow$	Same-pixel null $\downarrow$	AUC $\uparrow$
max width=	Ours	62.8%	48.3%	0.00%	0.628
	OpenDriveVLA	44.9%	35.7%	0.45%	0.542
	Impromptu-VLA	35.7%	36.4%	0.00%	0.478
	Alpamayo-R1	37.3%	49.3%	0.00%	0.395

Results and interpretation. Our model has the highest hazard–placebo AUC (0.628), exceeding OpenDriveVLA (0.542), Impromptu-VLA (0.478), and Alpamayo-R1 (0.395). The null response

is 0.00% for our model, Impromptu-VLA, and Alpamayo-R1, and 0.45% for OpenDriveVLA, indicating that the observed response rates are not explained by these measured null rates alone. Our model achieves the strongest hazard–placebo separation across the compared architectures, with an AUC advantage of 0.086 over OpenDriveVLA. The sky-placebo response rate of 48.3% identifies remaining sensitivity to irrelevant insertions (Sec. U). Together with trajectory accuracy and safety metrics, the shared-intervention results establish improved discrimination of obstacle relevance in the model’s planning response.

M Comparison between Next State Prediction and Current State Reconstruction

In Sec. H, when next state prediction is replaced with current state reconstruction, the average ADE difference is small at +0.01 m. To examine whether this small average difference means that the two models behave nearly identically across all scenes, or that the gap is large in specific scenes but cancels out in others, we quantitatively analyzed per-sample differences between the two models.

Analysis method. For each sample in the nuScenes validation set, we measure the per-sample 3-second endpoint L2 error of each model and define the difference as

$$\Delta = L_{\text{cur-state}}^2 - L_{\text{next-state}}^2.$$

Samples with $\Delta > 0$ are samples on which the next state prediction model is more accurate, and samples with $\Delta < 0$ are the opposite. To compare the two extremes of this distribution, we separate the top 25% ($\Delta \geq 0.094$ m, $n = 1,504$) as the *next-beneficial* group and the bottom 25% ($\Delta \leq -0.055$ m, $n = 1,504$) as the *cur-beneficial* group, and compare the driving scene characteristics of each group. The statistical significance of the differences is verified using the *Mann–Whitney U test*, a non-parametric test that does not require a normality assumption on the distribution.¹

We compare the six scene characteristics listed in Table 17. Each characteristic captures a different aspect of the driving situation. *Ego angular velocity* is the yaw rate of the ego vehicle, indicating the degree of changes in ego pose such as turning and lane changes. *Ego speed* is the magnitude of longitudinal motion; together, the two variables determine the dynamic level of the ego. *# front objects* quantifies scene complexity as the number of objects located in the front region of the ego, and *Min front distance* indicates collision risk as the distance to the nearest front object. *Nearest obj relative speed* is the relative speed of the nearest object with respect to the ego, measuring the intensity of dynamic interactions such as overtaking and merging. Finally, *Nearest agent turnover* is a binary indicator of whether the “nearest object” has changed to a different instance between the previous frame and the current frame, capturing the volatility of surrounding objects.

Results and interpretation. As shown in Table 17, among the six scene characteristics, the one that most strongly distinguishes the two groups is *ego angular velocity*. The mean angular velocity of the *next-beneficial* group is 0.072 rad/s, about 47% higher than that of the *cur-beneficial* group (0.049 rad/s), and the difference is statistically very strong with $p = 1.2 \times 10^{-14}$. *Nearest obj relative speed* also shows a large difference between the two groups ($p = 1.6 \times 10^{-2}$). On the other hand, ego speed, the number of front objects, the minimum front distance, and nearest agent turnover were essentially the same between the two groups (all with $p > 0.05$).

This result is intuitively interpretable. At the moment when the ego performs a turn or lane change, the front scene differs significantly from the current frame due to the change in the ego’s own yaw. Next state prediction informs the IK Network of the spatial layout of the scene after the turn in advance, whereas current state reconstruction by definition provides only the view *before* the turn, so during the turn the IK Network is fed stale visual context. That is, although the average ADE difference is small at +0.01 m, the gap is concentrated in the specific driving regime of turning, while the two models behave almost equally in straight-line driving. The per-scene analysis therefore locates NSP’s planning advantage in dynamic ego situations, where anticipating the future scene is especially relevant to trajectory generation.

¹The Mann–Whitney U test is a non-parametric method that tests whether two samples are drawn from the same distribution. Driving variables such as ego speed and angular velocity generally do not follow a normal distribution, so we use this test instead of the t-test, which requires distributional assumptions.

Table 17: Per-sample analysis of next state prediction’s benefit on nuScenes *val*. ***: $p < 0.001$, *: $p < 0.05$, n.s.: $p \geq 0.05$.

	Scene characteristic	next-benef.	cur-benef.	p -value	sig.
max width=	Ego angular velocity (rad/s)	0.072	0.049	1.2×10^{-14}	***
	Ego speed (m/s)	6.08	5.91	3.7×10^{-1}	n.s.
	# front objects	2.01	2.01	4.1×10^{-1}	n.s.
	Min front distance (m)	21.01	20.63	1.9×10^{-1}	n.s.
	Nearest obj. relative speed (m/s)	<u>2.87</u>	<u>3.28</u>	1.6×10^{-2}	*
	Nearest agent turnover (binary)	0.184	0.170	3.3×10^{-1}	n.s.

N Per-sample Correlation between Next State Prediction Quality and Trajectory Accuracy

The ablation in Sec. H showed the architectural contribution of the next visual state prediction (NSP) objective. Complementarily, in this section we verify, on a per-sample basis within a single trained model, whether samples with more accurate NSP also have more accurate trajectories. That is, while the previous section asks “does the objective help or not?”, this section asks “within the same model, how does sample-level NSP quality relate to trajectory quality?”

Setup. For all 5,869 ego tokens in the nuScenes validation set (every sample for which a next frame exists), we measure the following. NSP MSE is defined as the element-wise mean squared error between the model’s output next visual state latent $\hat{V}_{t+\Delta}$ and $\text{Proj}(V_{t+\Delta}^{\text{GT}})$, the GT next-frame DINOv3 visual feature projected into the same hidden space. The projector uses the trained model’s projector as is, so that the two representations are compared in the same space. NSP cosine is the average per-token cosine similarity. Plan L2@ k s is the Euclidean distance between the model’s k s endpoint and the GT.

Correlation results. As shown in Table 18, the per-sample correlation between NSP quality and plan accuracy is weak and the sign is opposite to intuition. Spearman ρ between NSP MSE and Plan L2@3 s is -0.144 ($p = 1.2 \times 10^{-28}$), and between NSP cosine and Plan L2@3 s is $+0.166$ ($p = 1.0 \times 10^{-37}$); both metrics suggest that the better the NSP fit, the slightly larger the plan L2. With a large sample size of $N = 5,869$, p values are extremely small, but the effect size is weak with $|\rho| < 0.17$. Plan L2 by NSP MSE quartile is also non-monotonic (L2@3 s is 1.471 m at Q1, the best NSP, and 1.160 m at Q4, the worst NSP). That is, whether the entire next visual state latent is accurate is not a per-sample determinant of trajectory accuracy.

Interpretation. Two properties of the prediction tasks explain why global reconstruction error and planning error measure different aspects of model behavior. First, Plan L2 and NSP MSE measure sample difficulty along different axes. The Plan L2 of a sample is largely determined by how predictable the GT trajectory itself is: for ego-dominated samples such as constant-velocity straight-line driving, the GT trajectory closely matches a simple ego-status extrapolation, so the Plan L2 is small for any reasonable trajectory model regardless of what features it relies on. The same sample, however, need not be trivial from the NSP perspective, since front-vehicle motion, signal changes, and background flow can cause the next visual state to vary substantially. That is, the two variables follow different sample-difficulty distributions. Second, the IK Network uses only a partial subspace of the next visual state. Our IK is a cross-attention diffusion that decodes only the trajectory-relevant subspace of the next visual state $\hat{V}_{t+\Delta}$ into a trajectory. Therefore, even if the global MSE is small, if the trajectory-relevant dimensions are inaccurate, the plan will be wrong, and vice versa.

The controlled ablations directly establish the contribution of NSP: replacing it with current-state reconstruction increases ADE from 0.06 to 0.07 m, with gains concentrated in dynamic regimes (Table 17), and reduces the hazard–placebo gap from 37.6 to 5.8 percentage points with the diffusion decoder fixed (Sec. I). Removing the IK Network separately increases ADE to 0.31 m (Sec. H). These results identify the value of supervising future visual content and decoding trajectories through the state interface. Global token reconstruction error within a trained model measures a different quantity from these architectural contributions.

Table 18: Per-sample correlation between Next State Prediction (NSP) quality and trajectory accuracy on nuScenes val split ($N = 5,869$). NSP MSE is computed in the projected hidden space (896-D); NSP cosine is the mean token-wise cosine similarity. A larger NSP MSE / smaller NSP cosine indicates a worse next-state prediction. ***: $p < 0.001$, **: $p < 0.01$, *: $p < 0.05$, n.s.: $p \geq 0.05$.

max width=						
NSP metric	Plan L2	Pearson r	p	Spearman ρ	p	
NSP MSE	@ 1 s	-0.013	3.4×10^{-1} n.s.	-0.092	1.9×10^{-12} ***	
	@ 2 s	-0.018	1.8×10^{-1} n.s.	-0.110	3.5×10^{-17} ***	
	@ 3 s	-0.025	5.3×10^{-2} n.s.	-0.144	1.2×10^{-28} ***	
	avg	-0.027	4.1×10^{-2} *	-0.139	1.5×10^{-26} ***	
NSP cosine	@ 1 s	+0.034	8.4×10^{-3} **	+0.108	1.2×10^{-16} ***	
	@ 2 s	+0.045	5.4×10^{-4} ***	+0.139	1.1×10^{-26} ***	
	@ 3 s	+0.061	2.6×10^{-6} ***	+0.166	1.0×10^{-37} ***	
	avg	+0.066	4.1×10^{-7} ***	+0.165	5.4×10^{-37} ***	

Plan L2 (m) averaged within each NSP MSE quartile. Trends are non-monotonic.						
	Quartile	NSP MSE	L2@1 s	L2@2 s	L2@3 s	L2 avg
max width=	Q1 (best NSP)	0.016	0.115	0.548	1.471	0.711
	Q2	0.018	0.143	0.463	1.397	0.667
	Q3	0.019	0.117	0.584	1.461	0.720
	Q4 (worst NSP)	0.022	0.103	0.400	1.160	0.554

O Stratified Evaluation by Ego-Status Difficulty in nuScenes

The average nuScenes ADE in Table 3 appears to saturate at the level of 0.06 m for our model, but this average does not imply uniform performance across all scenes. Li *et al.*[27] pointed out that a majority of nuScenes samples lie in an ego-dominated regime where trajectories can be extrapolated using only ego status, and that such samples dominate the average metric. In this section, we directly address this critique by stratifying samples according to the degree to which they can be followed using only ego status, and analyzing our model’s performance.

Stratification method. For each sample, we generate a trajectory by constant-velocity + constant-yaw-rate extrapolation using only ego status (longitudinal velocity v_x , lateral velocity v_y , yaw rate v_{yaw}), and define its ADE relative to the GT as the ego-status gap. We classify the top 30% of this gap (gap ≥ 2.14 m, $n = 1,805$) as the VisionCritical subset and the bottom 30% (gap ≤ 0.668 m, $n = 1,805$) as the *EgoDominated* subset. Intuitively, EgoDominated consists of scenes such as constant-velocity straight-line driving in which the trajectory is almost determined by ego status alone, while VisionCritical consists of scenes such as turns, lane changes, and front-vehicle responses for which visual scene understanding is essential.

Compared models. We compare three models: (i) the baseline OpenDriveVLA [52], (ii) a variant that architecturally removes the visual stream from the baseline and is trained using only ego status and text commands (*EgoTextOnly*), and (iii) our model (*Ours*). *EgoTextOnly* serves as the extreme limit baseline of an ego-shortcut that cannot use vision.

Results. Table 19 yields the following three observations.

(1) Quantitative validity of the stratification. The more a baseline cannot rely on vision, the larger its ADE degradation on VisionCritical: OpenDriveVLA degrades from 0.166 m on EgoDominated to 0.429 m on VisionCritical (+158%), while our model, which actively exploits vision, only degrades from 0.049 m to 0.066 m (+34%). EgoTextOnly (+95%) lies between the two models, showing a monotonic pattern of DriveVLA-orig $\rightarrow$ EgoTextOnly $\rightarrow$ Ours. This is a sanity check showing that the stratification is actually separating “the necessity of vision”.

(2) The improvement of Ours is concentrated in VisionCritical. Compared to DriveVLA-orig, the absolute improvement of our model is -0.117 m (-71%) on EgoDominated, while on VisionCritical it is -0.363 m (-85%), about 3 $\times$ larger. That is, the overall average ADE of 0.06 m has the effect of compressing the average because both models converge close to the kinematic ceiling on the

Table 19: **Stratified evaluation on nuScenes val** ($n = 6019$) **by ego-status difficulty**, following the criterion of Li *et al.* [27]. For each sample, we measure the GT-relative ADE of an ego-status-only constant-velocity + constant-yaw-rate extrapolation (*ego-status gap*); the top 30% (≥ 2.14 m, $n = 1,805$) forms the *VisionCritical* subset and the bottom 30% (≤ 0.668 m, $n = 1,805$) forms the *EgoDominated* subset. We compare three models: the original OpenDriveVLA [52] (*DriveVLA-orig*), a text-only variant trained with the visual stream architecturally removed (*TextOnly-novis*), and our model (*Ours*). We report UniAD avg. L2 (m) and Collision (%); **bold** = best per column.

	Subset	DriveVLA-orig		TextOnly-novis		Ours	
		L2↓	Coll.↓	L2↓	Coll.↓	L2↓	Coll.↓
max width=	Full ($n=6,019$)	0.347	0.090	0.308	0.078	0.063	0.044
	EgoDominated ($n=1,805$)	0.166	0.012	0.190	0.003	0.049	0.060
	VisionCritical ($n=1,805$)	0.429	0.172	0.371	0.078	0.066	0.011

ego-dominated group, while the true gap between models is clearly revealed on the vision-critical subset at the level of 0.36 m.

(3) **Agreement with collision metrics.** The same pattern is also observed in the collision rate, which is independent of trajectory ADE. On the VisionCritical subset, DriveVLA-orig has a collision rate of 0.172% while our model has 0.011%, about $15\times$ lower. On EgoDominated, all three models converge to a very low collision rate of less than 0.1%, and this region consists by definition of straight-line, constant-velocity, collision-trivial scenes where the differences between models are not meaningful. Since collision avoidance requires accurate perception of visual obstacles such as pedestrians and vehicles, the large gap on VisionCritical is consistent with the observation in Sec. P.1 that our model’s visual grounding plays a decisive role in collision-critical scenes.

P GradCAM Visual Grounding Analysis and Methodology

This section summarizes the detailed measurement protocol for the visual grounding analysis in Sec. P.1. We adapt the backpropagation target and hook layer to each model’s trajectory output form and report common object-overlap metrics. The overlap metrics quantify alignment between model sensitivity and scene objects, complementing the direct behavioral evidence from shared hazard, placebo, and null interventions in Sec. L. The backpropagation protocol below specifies the architecture-dependent meaning of each saliency target.

P.1 Visual Grounding Results

The stitching experiment in Sec. 5.3.1 shows that visual perturbations *change* the trajectory; here we examine where gradient-based saliency is concentrated. On the nuScenes validation set, we measure how much each model’s GradCAM [36] overlaps with on-road objects (vehicles and pedestrians). For each sample, we project GT 3D bounding boxes from the front camera onto the 2D image plane to obtain an object mask M , and obtain a per-sample saliency map S using the architecture-specific backpropagation targets described in Sec. P.3. We report three complementary metrics: (1) *Pointing Mass* (PM), the fraction of CAM energy that falls inside M ; (2) *Average Precision* (AP), a threshold-free ranking score treating M as positives and S as scores; and (3) *IoU/F1 @ top-20%*, region-level overlap on the top-20% saliency pixels. The following subsections detail the measurement protocol. We compare against OpenDriveVLA [52], Alpamayo-R1 [44], and Impromptu-VLA [2].

Table 20: Visual grounding via GradCAM–object overlap on nuScenes val set.

Model	Pointing Mass↑	Avg. Precision↑	IoU@top20↑	F1@top20↑
Impromptu-VLA 3B [2]	0.103	0.109	0.067	0.121
Alpamayo-R1 10B [44]	0.178	0.202	0.093	0.161
OpenDriveVLA 0.5B [52]	<u>0.205</u>	<u>0.243</u>	<u>0.105</u>	<u>0.177</u>
Ours	0.268	0.281	0.145	0.236

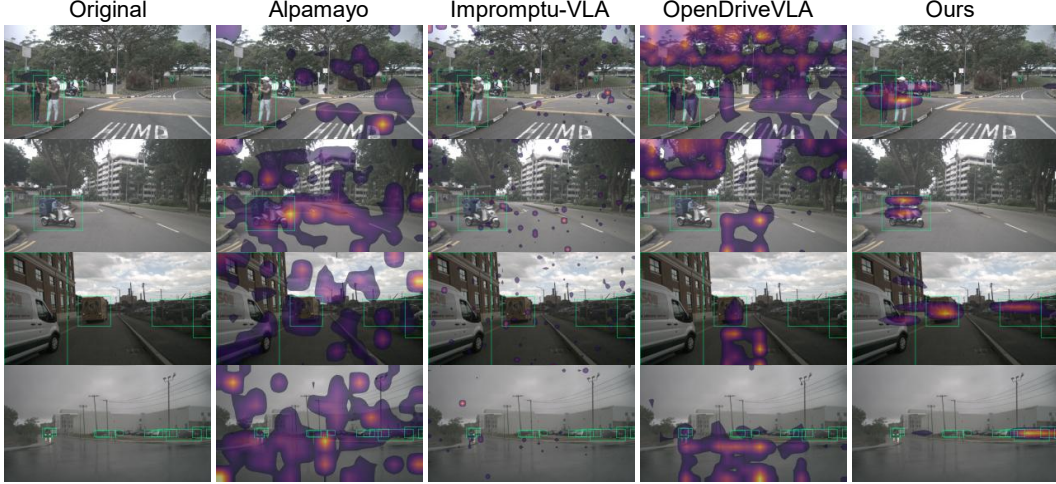


Figure 4: Qualitative comparison of GradCAM saliency on nuScenes *val set*. Each row corresponds to a single driving scene, and each column shows the GradCAM of Alpamayo-R1 [44], Impromptu-VLA [2], OpenDriveVLA [52], and our model overlaid on the input image for that scene.

Results. As shown in Table 20, our method achieves the highest object alignment on all three metrics. In Pointing Mass we record 0.268, +31% over the second-place OpenDriveVLA (0.205) and up to +160% over Impromptu-VLA (0.103); on AP we obtain 0.281 (+16% over OpenDriveVLA), and the 95% bootstrap confidence intervals do not overlap. Fig. 4 confirms this qualitatively: existing models disperse GradCAM over road surface, sky, and background, while ours concentrates it on planning-relevant objects such as front vehicles and pedestrians, consistent with the analysis in Sec. 3.

P.2 Metric Definitions

Given a per-sample saliency map $S \in \mathbb{R}_{\geq 0}^{H \times W}$ and an object mask $M \in \{0, 1\}^{H \times W}$ obtained by projecting GT 3D bounding boxes (categories `vehicle.*` and `human.pedestrian.*`, with boxes containing fewer than one LiDAR point excluded) onto the 2D image plane, we report three complementary metrics.

(1) Pointing Mass is a *threshold-free* metric measuring how much of the model’s “attention budget” is allocated to actual object regions, defined as the ratio of CAM energy located inside the object mask to the total CAM energy:

$$\text{PM} = \frac{\sum_{(i,j) \in M} S_{ij}}{\sum_{(i,j)} S_{ij}}.$$

This metric measures only “*where the model is looking*” regardless of the absolute scale of the saliency map.

(2) Average Precision (AP) is the precision-recall AUC averaged over all thresholds, treating the object mask as positive labels and S as the ranking score. Since no threshold is artificially fixed, it answers the question “are pixels with higher saliency values more likely to actually lie on objects?”

(3) IoU & F1 @ top-20% treats the top-20% pixels of the saliency map (i.e., pixels above the per-image 80-percentile p_{80}) as binary positives and measures spatial overlap with the object mask in terms of IoU and F1, complementing the “ranking consistency” captured by PM and AP with a region-level agreement.

The three metrics are based on different assumptions (energy ratio, threshold-free ranking, hard mask overlap), so a consistent advantage across all metrics indicates an essential improvement in grounding quality rather than single-metric hacking.

Scale invariance. The raw gradient magnitudes across different architectures cannot be directly compared, so all three metrics above are constructed to be robust to cross-architecture gradient scale differences: (i) PM is a ratio of sums and is therefore invariant to multiplication by a positive

constant; (ii) AP is a rank-based score and is invariant to any positive monotonic transformation of S ; (iii) IoU/F1 @ top-20% uses the per-image 80-percentile as the threshold, so the binarization is unchanged under multiplication by a positive constant. In addition, all CAMs are min-max normalized per camera per image to the range $[0, 1]$ before being used in metric computation.

P.3 Backpropagation Target

GradCAM uses the gradient of some scalar S with respect to the visual token input as visual saliency, so the choice of S determines the meaning of the measurement. Where available, we use the model’s native trajectory supervision objective as the backpropagation scalar; for Alpamayo-R1, we use the proxy described below. Each scalar measures sensitivity in its model’s computation; comparisons use the normalized spatial-overlap metrics above, with shared input interventions providing the common behavioral evaluation.

Ours (CrossAttentionDiffusionIK). Instead of an ELBO or marginal log-likelihood, we evaluate the standard DDPM denoising loss used during training as a single Monte Carlo sample:

$$\mathcal{S}_{\text{ours}} = -\left\| \hat{\epsilon}_{\theta}(x_t, t, V_t, \hat{V}_{t+\Delta}) - \epsilon \right\|_2^2, \quad (32)$$

with $x_t = \sqrt{\alpha_t} T^* + \sqrt{1 - \alpha_t} \epsilon$, $t = T/2 = 25$ (the mid-noise level of the training/inference horizon $T = 50$), $\epsilon \sim \mathcal{N}(0, I_{12})$ with a deterministic seed, and $T^* \in \mathbb{R}^{12}$ the GT 6-step ego trajectory normalized by trajectory scale $s = 20$. The conditioning of $\hat{\epsilon}_{\theta}$ consists of (a) current visual state tokens V_t (DINOv3 cache, 540×896) and (b) the next visual state latent $\hat{V}_{t+\Delta}$ predicted by the LLM (1 s/2 s/3 s multi-horizon). $\mathcal{S}_{\text{ours}}$ is identical to $\lambda_{\text{traj}} \mathcal{L}_{\text{traj}}$ used during training, and the gradient of a single (t, ϵ) sample is an unbiased estimator of the full training loss gradient.

Token-output baselines (OpenDriveVLA, Impromptu-VLA). For models in which the LLM emits the trajectory as a text token sequence, we use the token-level cross-entropy loss with respect to the GT trajectory tokens used during training as is: $\mathcal{S}_{\text{tok}} = -\text{CE}(\text{LLM logits}, \text{GT trajectory tokens})$. This matches the native training supervision of those models.

Diffusion-only baseline (Alpamayo-R1). Alpamayo-R1 architecturally lacks token-level GT trajectory supervision and uses only a diffusion action head. In this case, among differentiable scalars over LLM hidden states, we use the logit sum at the `<traj_future_start>` trigger position closest to the start of trajectory generation as a proxy. This is not a strict log-likelihood, and measures how much visual context affects the LLM’s representation right before trajectory generation.

P.4 Hook Layer Selection

For all models, we hook at the same location: the LLM 1st decoder layer pre-hook, i.e., the point where the visual representation has just entered the LLM after the vision encoder (and the projector, when present). This unification ensures that all GradCAM measurements compare gradients at a structurally equivalent stage of the network.

Q Evaluate Position Awareness in the Stitching Response

To separate whether the trajectory shortening reported in Sec. 5.3.1 originates from grounding on the positional meaning of the obstacle or merely from a response to the perceptual properties of the sprite, we compare five stitching variants in which the position (road horizon vs. sky) and size (scale $0.25 \sim 1.8$) of the sprite are independently varied (Table 21). All measurements use paired same-seed sampling (the trajectories before and after stitching share the same DDIM noise), with $K = 50$ samples averaged per token, so that Δ reflects the stitching-induced change rather than sampling stochasticity.

The signed mean Δ scales monotonically with both the realism of the obstacle and its size: Near (road, scale 1.8) yields -1.04 m, Far (road horizon, scale 0.5) yields -0.54 m, while sprites placed in the sky (Sky, scale 0.30; SkyFar, scale 0.50) have signed means within ± 0.10 m and the sample-level reaction is small ($|\Delta| < 1$ m on 77.6% and 63.2% of scene frames, respectively). The decisive test is the paired control between Far and SkyFar, which share the same sprite scale (0.5) and differ only in position: for the same frame, **65.5%** of samples show a larger response when the sprite lies on the road than when it lies in the sky, with a signed paired mean of $\Delta_{\text{Far}} - \Delta_{\text{SkyFar}} = -0.64$ m. A conditional

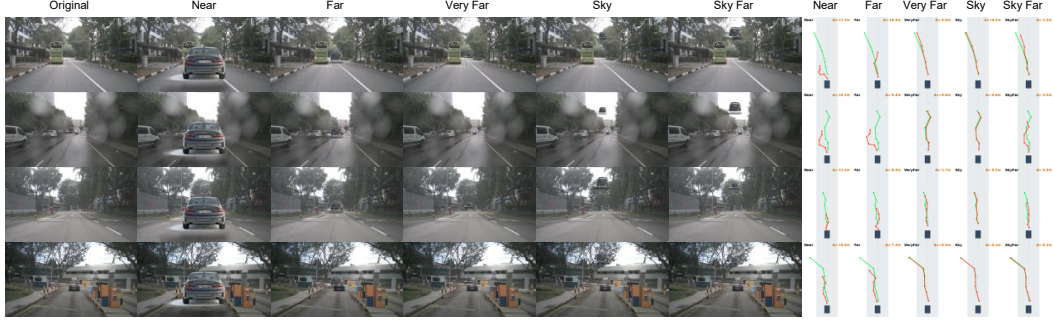


Figure 5: Qualitative examples of the position-size stitching variants on nuScenes *val*. For each scene, we visualize the predicted trajectory before (green) and after (red) stitching a vehicle sprite at four placements: Near and Far on the road, and Sky and SkyFar in the sky region (Far and SkyFar share the same sprite scale). The trajectory visibly bends or shortens for road-placed sprites, while sky-placed sprites of the same size leave the trajectory essentially unchanged, consistent with the quantitative results in Table 21.

analysis on the 1,442 scene frames for which *Near* triggers a strong slowdown ($\Delta < -3$ m) further supports this: under the same scene frames, the model essentially ignores the Sky and Very-Far variants ($|\Delta| < 1$ m on 76% and 74% of those tokens), while still reacting on Far ($|\Delta| < 1$ m on only 35%).

These results indicate that the response is genuinely position-aware rather than a generic size-based reaction: when sprite size is held fixed, the model distinguishes between road and sky, and conditional on a strong on-road reaction it correctly suppresses the response for off-road placements. Fig. 5 illustrates this qualitatively: trajectories visibly bend or shorten in response to road-placed sprites (Near, Far), while sky-placed sprites of the same size (Sky, SkyFar) leave the trajectory essentially unchanged.

Table 21: Stitching response on nuScenes *val* split ($n = 6019$), decomposed by sprite size and position, measured with paired same-seed sampling (K=50 DDIM mean). Δ is the signed 3 s endpoint displacement (after – before, in meters); $|\Delta|$ is its magnitude. Far and SkyFar share sprite scale (0.5) and differ only in position (road horizon vs. sky), forming the decisive paired control.

Stitching	scale	position	mean Δ (m)	mean $ \Delta $ (m)	median $ \Delta $ (m)	$ \Delta < 1$ m (%)
Near (Sec. 5.3.1)	1.8	ego front (road)	−1.04	2.51	1.95	27.8
Far	0.5	road horizon	−0.54	1.81	1.20	44.0
Very-Far	0.25	horizon (small)	−0.09	0.74	0.43	77.0
Sky	0.30	sky (small)	−0.00	0.72	0.44	77.6
Sky-Far	0.5	sky	+0.10	1.04	0.70	63.2

Far vs. *SkyFar* paired control ($n = 6019$, same scale 0.5, position only):

$|\Delta_{\text{Far}}| > |\Delta_{\text{SkyFar}}|$: 65.5% signed paired mean $\Delta_{\text{Far}} - \Delta_{\text{SkyFar}} = -0.64$ m

R Contribution of QA Co-Training to Trajectory Planning

Iteration-matched ablation. To evaluate whether the question-answering component benefits planning, we remove QA multi-task supervision entirely and retrain with the number of optimization iterations matched to the full model (140,000 steps). Both models are evaluated on the full nuScenes validation set ($n = 6,019$) using the ST-P3 planning metrics. This comparison controls the number of optimization steps, so the QA model’s advantage cannot be explained simply by a larger iteration budget. The controlled quantity is the optimization-step budget; QA supplies the additional supervision being evaluated.

Removing QA increases average L2 error by 0.32 m, from 0.06 to 0.38 m, and doubles the reported collision rate from 0.09% to 0.18%. QA co-training therefore improves both trajectory accuracy and collision rate under the matched iteration budget.

Table 22: Effect of QA co-training on nuScenes validation ($n = 6,019$). Both models are trained for 140,000 optimization steps. Lower is better for both ST-P3 metrics.

Model	L2 avg. (m) ↓	Collision avg. (%) ↓
Ours (with QA co-training)	0.06	0.09
QA removed (iterations matched)	0.38	0.18

Table 23: Performance on nu-Caption. **Bold**=best, Underline=second.

Method	BL-1	BL-2	BL-3	BL-4	BERT-S
Mini-GPT4	15.0	6.8	3.7	2.6	84.4
Instruct-BLIP	18.7	13.4	7.4	5.2	85.9
LLaVA1.5	20.0	12.1	8.6	5.4	85.0
LLaMA-AdapV2	30.2	17.3	10.4	7.5	86.5
LiDAR-LLM	41.0	30.0	23.4	19.3	91.3
OpenDriveVLA-0.5B	<u>47.2</u>	<u>35.8</u>	<u>29.4</u>	<u>25.2</u>	<u>91.9</u>
Ours-0.5B	51.2	39.0	31.8	26.9	97.6

Table 24: Performance on Nu-X. **Bold**=best, Underline=second.

Method	CIDEr	BLEU-4	METEOR	ROUGE-L
Hint-UniAD	21.7	4.2	12.7	27.0
Hint-VAD	22.4	4.2	<u>13.2</u>	<u>27.6</u>
GPT-4o	19.0	4.0	10.3	24.9
Gemini 1.5	17.6	3.4	9.3	23.4
Vote2Cap-DETR	15.3	2.6	10.9	24.2
TOD ³ Cap	14.5	2.5	10.5	23.5
OpenDriveVLA-0.5B	<u>32.3</u>	<u>5.4</u>	12.5	27.9
Ours-0.5B	35.2	5.8	20.2	26.2

Training-time transfer through the predicted state. QA provides auxiliary supervision to the shared LLM during training. At planning inference, the IK Network consumes only the current and predicted future visual states, $(V_t, \hat{V}_{t+\Delta})$; generated QA answers are not inputs to the trajectory decoder. QA updates can therefore improve the representation used to predict $\hat{V}_{t+\Delta}$, through which the shared LLM influences the trajectory. QA transfers to planning through the shared representation learned during training, with no generated answer in the inference-time trajectory path. The route command remains an upstream input to future-state prediction, as distinguished from QA outputs in Sec. K.

The iteration-matched ablation establishes QA co-training as a substantial contributor to planning performance, consistent with its role in learning the shared future-state representation. The shared hazard-placebo evaluation and conditioning diagnostics provide complementary evidence about visual use (Secs. I and L). These results also make QA co-training part of the explanation for the full model’s planning performance, alongside NSP and the IK decoder.

S Question and Answering Performance.

S.1 Experiments on nu-Caption.

As shown in Table 23, our method also consistently outperforms the baselines, including OpenDriveVLA, on all metrics in driving scene captioning. Specifically, BLEU-1 through BLEU-4 are recorded as 50.4, 38.4, 31.2, and 26.4, respectively, showing consistent improvements of +3.2, +2.6, +1.8, and +1.2 over OpenDriveVLA [52]. In particular, the BERT-Score, which measures semantic similarity, reaches 97.6, a large gap of +5.7 over OpenDriveVLA (91.9), suggesting that beyond mere n-gram matching, the LLM understands and describes the driving scene more accurately at the semantic level. This result supports that our training strategy, which forces attention onto visual tokens through next visual state prediction, recovers not only trajectory planning but also scene understanding ability itself.

S.2 Experiments on nu-X.

As shown in Table 24, our method also shows improvements over the baseline on three out of four metrics on the nu-X benchmark, which requires reasoning about driving scenes. On CIDEr, we achieve 35.1, an improvement of +2.8 over OpenDriveVLA [52], and on BLEU-4 we record 5.6, an improvement of +0.2. In particular, on METEOR we achieve 20.0, a large gap of +6.8 over Hint-VAD [6] (13.2), which means that our model produces answers that are semantically richer and more aligned with the reference descriptions. On ROUGE-L, we record 26.1, slightly below OpenDriveVLA (27.9), but considering the large advantage on CIDEr-METEOR, which together

Table 25: Performance on nuScenes-QA. **Bold**=best, Underline=second.

Method	Ext	Cnt	Obj	Sts	Cmp	H0	H1	Acc
LLaMA-AdapV2	19.3	2.7	7.6	10.8	1.6	15.1	4.8	9.6
LLaVA1.5	45.8	7.7	7.8	9.0	52.1	25.7	41.5	26.2
LiDAR-LLM	74.5	15.0	37.8	45.9	57.8	-	-	48.6
BEVDet+BUTD	<u>83.7</u>	<u>20.9</u>	<u>48.8</u>	52.0	<u>67.7</u>	-	-	<u>57.0</u>
OpenDriveVLA-0.5B	83.9	22.0	50.2	57.0	68.4	62.3	56.5	58.4
Ours-0.5B	83.7	19.5	47.1	<u>52.8</u>	67.3	<u>59.9</u>	<u>54.7</u>	56.4

comprehensively reflect overall caption quality, we can confirm that our model performs more meaningful reasoning about driving scenes.

S.3 Experiments on nuScenes-QA.

As shown in Table 25, on nuScenes-QA our method shows the second-best performance, slightly below OpenDriveVLA [52], but the gap is only 1.5%p in overall accuracy (58.4 $\rightarrow$ 56.9), which is essentially comaparable to BEVDet+BUTD [33] (57.0). In detailed categories, we also show nearly the same performance as OpenDriveVLA on *Existence* (83.8 vs. 83.9) and *Comparison* (67.8 vs. 68.4), and show relatively larger gaps in categories that require fine-grained object identification, such as *Counting* (-1.5), *Object* (-3.1), and *Status* (-2.7).

This performance gap can be explained by the difference in visual representation between our method and OpenDriveVLA. Since OpenDriveVLA leverages a UniAD BEV encoder [15] pretrained on nuScenes, it operates on top of a top-down representation in which the spatial layout of object instances is explicitly encoded. This is naturally favorable for the Counting/Object/Status categories of nuScenes-QA, which require accurate instance-level identification and spatial relations, such as “*how many vehicles are visible?*” or “*what is the status of the object on the right of the ego?*”. In contrast, in order to generalize to environments without a publicly available BEV encoder, such as NAVSIM, our method intentionally adopts a perspective-view feature (DINOv3), and the next state prediction objective is also designed to learn planning-relevant scene dynamics rather than fine-grained object identification. That is, our method pays a slight cost in instance-level perception relative to BEV-based models, but in exchange obtains consistent and large improvements on the essential tasks of a driving VLA, planning and high-level scene understanding. Moreover, the gap on nuScenes-QA (overall -1.5%p) itself stays at the level of BEVDet+BUTD, so our method does not significantly sacrifice perception ability; rather, it shows only a small gap on the instance-level accuracy that BEV pretraining provides.

T Inference Speed

Setup. We measure end-to-end inference latency of our 0.5B model on a single NVIDIA RTX A6000 over the nuScenes *val set* with three front cameras. Following common practice in VLA latency reporting, we decompose the forward pass into (i) the visual encoder (DINOv3 ViT-S/16 over 3 cameras at 288×160) and (ii) the LLM together with the IK Network (50-step DDPM sampling). Each stage is timed independently with `torch.cuda.synchronize()` after 10–20 warm-up samples.

Stage breakdown and end-to-end latency. Table 26 summarises the per-stage latency. The vision encoder takes 20.7 ms on average and the LLM+IK stage takes 323.7 ms, giving an end-to-end latency of ~ 344 ms (2.9 FPS) when the camera stream is fed directly to GPU memory (the ~ 515 ms JPEG disk-load cost in our offline benchmark is an artefact of file I/O and does not appear in a real ISP $\rightarrow$ GPU pipeline).

Comparison to other VLAs. It is worth noting that our ~ 344 ms is achieved on an RTX A6000, a workstation-class GPU that is markedly slower than the data-center A100/H100s used by the baselines below; despite this hardware disadvantage, our model has competitive latency. At the same 0.5B backbone, OpenDriveVLA reports 1.36 s on A100 (bf16), so our model is roughly $4\times$ faster while running on the slower GPU. At the VLA category level, mainstream Driving VLAs (DriveVLM, OpenDriveVLA-3B/7B, AutoVLA) sit around 1–2 s on A100, and large general-purpose VLMs (Qwen-2.5-VL-7B, LLaMA-3.2-11B/90B, GPT-4o, GPT-5) range from several seconds to tens of

Table 26: Inference speed of our 0.5B model on a single NVIDIA RTX A6000, measured on the nuScenes validation set with three front cameras and DDPM 50-step IK sampling. End-to-end is the GPU-only sum (vision encoder + LLM + IK). For context, we list reported latency of representative Driving VLAs and large VLMs.

Model / stage	Hardware	Latency	Throughput
<i>Per-stage breakdown of our model</i>			
Vision encoder (DINOv3 ViT-S/16, 3 cam)	A6000 (fp16)	20.7 ms	–
LLM (Qwen2.5-0.5B) + IK diffusion (50 steps)	A6000 (fp16)	323.7 ms	–
End-to-end (vision + LLM + IK)	A6000 (fp16)	~ 344 ms	2.9 FPS
<i>Reported latency of other Driving VLAs and VLMs</i>			
VLA-MP [14]	RTX 3090	125 ms	8.0 FPS
OpenDriveVLA-0.5B [52]	A100 (bf16)	1.36 s	0.74 FPS
OpenDriveVLA-3B [52]	A100 (bf16)	1.85 s	0.54 FPS
OpenDriveVLA-7B [52]	A100 (bf16)	1.74 s	0.57 FPS
DriveVLM [41]	–	~ 1.5 s	0.67 FPS
AutoVLA [53]	–	~ 1.0 s	~ 1 FPS
Qwen-2.5-VL-7B [48]	H100	8.8 s	0.11 FPS
LLaMA-3.2-11B [8]	H100	7.5 s	0.13 FPS
GPT-4o (closed)	API	12.1 s	0.083 FPS

seconds even on H100 or larger clusters. Sub-500 ms reports in the VLA literature are rare; ours is one of the few in this range, alongside specialised systems such as VLA-MP (125 ms on RTX 3090). This suggests that, on top of the accuracy gains in the main paper, the proposed architecture is also a practical choice for latency-sensitive deployment. The dominant cost is the 50-step DDPM sampling inside the IK Network, leaving room for further reduction via fewer diffusion steps, distillation, or visual-token pruning.

U Limitations and Future Work.

We have not compared the LLM future-state generator with a matched non-language alternative, or jointly removed ego status and command in a factorial experiment. These comparisons would extend the demonstrated input-level contributions to backbone selection and ego/command interaction effects (Sec. K).

Our method shows somewhat lower performance than BEV-based VLAs on a few categories of nuScenes-QA that require instance-level perception (Counting, Object, etc.), which is the result of a trade-off in which we intentionally chose a perspective-view representation in order to generalize to environments where no pretrained BEV encoder is provided (e.g., NAVSIM).

In addition, while our paired stitching analysis (Sec. Q) shows that the recovered visual grounding is genuinely position-aware (Far vs. SkyFar discrimination at 65.5%, paired mean -0.64 m), the precision of this grounding still has room to grow—the magnitude of the response also scales with sprite size, and a fraction of off-road placements still elicit non-trivial reactions. A natural direction for sharpening this signal is supervision that explicitly disentangles the position, size, and meaning of obstacles (e.g., using the ego-frame coordinates of synthesized obstacles as auxiliary outputs), and additional research in this direction is needed.

Moreover, our experiments are limited to open-loop and short-term closed-loop benchmarks. Validation on long-term closed-loop environments such as Bench2Drive [18], and extending the inverse-kinematics redefinition of this work to other end-to-end embodied policies beyond autonomous driving, would be interesting follow-up directions.